

Graduate Machine Learning - Homework 3

Kernel and Non-Kernel Learning Under Real-World Data Imperfections

Complete Expanded Technical Report with Full Workflow and Written Analysis

This expanded report explains the full workflow, code logic, generated results, figures, and interpretations for Dataset 1 through Dataset 4. The design follows the assignment requirement that 60% of the grade depends on data engineering, investigation, and deep analysis, while 40% depends on algorithmic implementation and modeling.

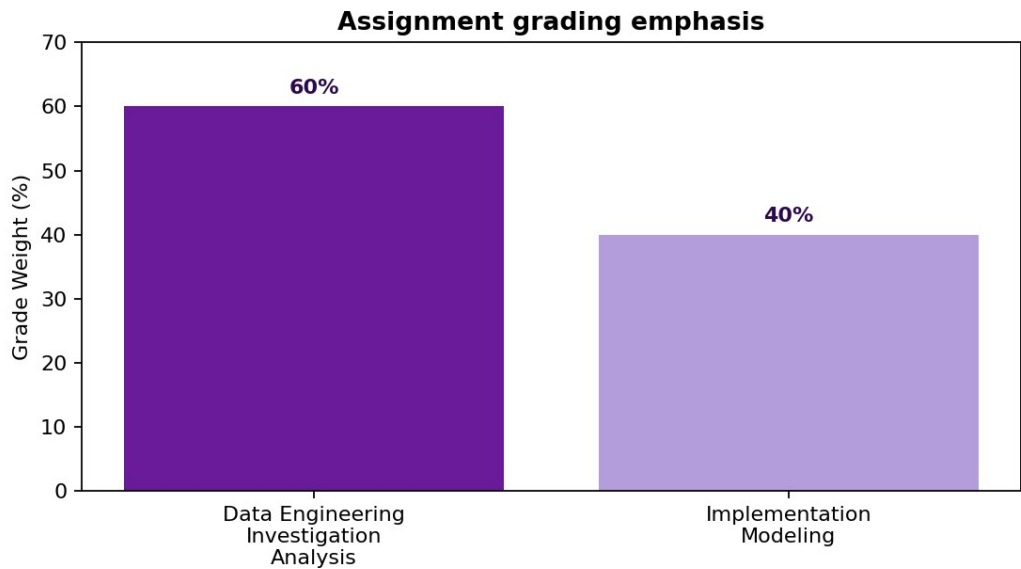


Figure 0. Assignment grading emphasis: the project is primarily a data-quality and reasoning assignment, not a pure accuracy contest.

This chart frames the entire report. The notebooks therefore spend substantial effort on missingness, leakage, outliers, feature quality, failure analysis, and bias before presenting model metrics.

0. Full Project Workflow: How the Four Notebooks Answer the Assignment

This section was added as a reading guide for the report. The assignment is not mainly asking for a high score table. It asks us to prove that real-world data imperfections control model behavior. Therefore, the work was organized around a repeated but dataset-specific pipeline: understand the raw data, diagnose data quality, engineer task-appropriate features, prevent leakage, implement models from scratch, compare kernel and non-kernel behavior, and then interpret failures and bias. The figures later in the report should be read as evidence for these decisions, not as decorative plots.

Connection to the assignment: The PDF states that 60% of the grade is Data Engineering, Investigation, and Deep Analysis, while only 40% is Algorithmic Implementation and Modeling. For this reason, every notebook first spends many sections on missing values, outliers, feature quality, label quality, and leakage before moving to linear, logistic, KNN, tree, and kernel models. This is why the report explains the workflow before listing final metrics.

0.1 What the Professor Wanted and How the Code Responds

Assignment demand	How it was implemented in the notebooks	Why it matters for the report
Do not use one universal pipeline for all datasets.	Each notebook has a dataset-specific engineering stage: Airbnb uses city-aware marketplace features; NYC 311 uses intelligent sampling, duplicate handling, and temporal service features; HR uses imbalance treatment and root-driver	This proves that the same model family behaves differently depending on topology, label quality, and operational data generation.

ML Homework 3 - Expanded Four-Dataset Report

	analysis; Online Retail converts invoices into customer-level RFM behavior.	
Investigate missing data as MCAR/MAR/MNAR.	The notebooks compute missing rates, missing indicators, association tests, and domain-based missingness classifications before imputation. Airbnb and Retail required the most explicit missingness reasoning; HR had almost no observed missingness; NYC 311 had operational missingness tied to agency/service fields.	This prevents blind mean-imputation and gives a mathematical/domain justification for imputation choices.
Compare Z-score, IQR, and Isolation Forest.	Each notebook applies Z-score and IQR as univariate methods and a NumPy-based Simple Isolation Forest as a multivariate anomaly detector. Outlier flags are used for analysis, while leakage-prone global flags are removed from final modeling inputs.	This shows the difference between single-column extremes and unusual multi-feature profiles, especially in long-tailed Airbnb and Retail distributions.
Identify redundant features, leakage, and near-constant variables.	The notebooks build feature inventories, detect constant/near-constant variables, high-cardinality identifiers, correlated numeric pairs, and target-derived leakage. Final candidate feature lists are task-specific.	This protects model evaluation from artificial performance caused by target leakage or memorization.
Implement core models from scratch.	The code implements Linear Regression, Logistic Regression, KNN, Decision Tree, Kernel SVM, Kernel Ridge Regression, Kernel KNN, and KPCA + classifier using NumPy/SciPy style logic rather than scikit-learn.	This satisfies the algorithmic implementation requirement while keeping the main focus on data imperfections.
Provide kernel investigation, not just metrics.	The report explains when linear, polynomial, and RBF kernels help or fail depending on locality, nonlinearity, dimensionality, noise, sparsity, and sample size.	This answers the assignment requirement that kernel success must be justified mathematically and dataset-specifically.
Perform failure, bias, computation, and theory-vs-reality discussion.	Each notebook isolates bad predictions or hard cases, evaluates group-level reliability, discusses kernel matrix cost, and compares real-world data with textbook datasets.	This converts the notebooks into a research-style report instead of a raw code appendix.

0.2 The Repeated End-to-End Code Pipeline

Although each dataset needed a different engineering strategy, the notebooks followed the same scientific structure. The purpose of the structure was reproducibility: every transformation creates an intermediate table, every major decision is saved, and every modeling dataset is separated into task-specific features and targets.

Pipeline stage	What the code does	Written interpretation added to the report
1. Load and inspect raw data	Read CSV, ZIP, or Excel files; print shape, column names, dtypes, missing counts, and sample rows.	Defines the raw topology and shows why the dataset cannot be modeled immediately.
2. Build targets	Create clean regression and classification labels such as price_numeric, resolution_time_hours, attrition_binary, future_spending, and future segment IDs.	Explains what the prediction problem actually is and removes rows with invalid or missing labels.
3. Data quality diagnosis	Compute missingness tables, duplicate checks, invalid value rules, missingness indicators, and missingness mechanism evidence.	Justifies whether values should be dropped, imputed, transformed, or represented with indicator features.
4. Feature engineering	Create domain-specific features: text indicators and city variables for Airbnb, temporal service features for NYC 311, HR risk flags and ratios for attrition, and RFM/customer history features for retail.	Shows how raw columns are converted into predictive signals that match each domain.
5. Outlier investigation	Compare Z-score, IQR, and Simple Isolation Forest, then preserve analysis flags and use log/winsorized versions only when leakage-safe.	Explains whether extreme values are data errors, rare valid observations, or meaningful high-risk profiles.
6. Feature quality and leakage audit	Remove identifiers, target-derived columns, global outlier flags, near-constant columns, and overly high-cardinality raw text fields.	Ensures that metrics measure generalization rather than memorization or leakage.
7. Train/test split and preprocessing	Split data with NumPy, use stratification where needed, fit imputers/scalers/encoders only on training data, and transform test data with training parameters.	Prevents preprocessing leakage and makes evaluation realistic.
8. From-scratch modeling	Train non-kernel and kernel models, compute regression/classification metrics from scratch, and compare results.	Shows how model families respond to the engineered manifold.

9. Failure and bias analysis	Inspect worst predictions, subgroup metrics, geography/country/department bias, class imbalance, and time/channel effects.	Turns errors into explanations and identifies production risks.
10. Save outputs and report artifacts	Save CSV tables, model summaries, figures, final arrays, and report-ready text blocks.	Makes the result reproducible and turns notebooks into a formal mini-conference submission.

0.3 Dataset-by-Dataset Workflow Before Reading the Results

The following four workflow summaries explain what each notebook did before the charts and result tables. They are placed at the beginning because the results later in the report only make sense if the data process is clear.

Dataset 1 workflow - Airbnb five-city marketplace data

What the assignment wanted: Airbnb required price regression and superhost classification. It specifically required justification of missing-value imputation, criteria for dropping features, and defense of outlier handling. The code first expanded the original single-city setup into five city datasets: Amsterdam, Barcelona, Chicago, New York, and Madrid. This was important because city is not a small categorical detail; it changes the distribution of prices, missingness, property types, and host behavior.

- The loader reads multiple city files, adds a city column, and unions schemas because different cities do not always expose exactly the same columns.
- The target-cleaning stage converts price strings into numeric price values and converts `host_is_superhost` into a binary classification target.
- Text fields such as `description`, `neighborhood_overview`, `host_about`, and `amenities` are not imputed as text. Instead, the code creates `has_text` indicators and text-length features, because the presence or absence of text itself is informative.
- Missingness is analyzed column-by-column and city-by-city. This matters because a global imputation rule can hide city-specific data-collection patterns.
- Outliers are handled carefully: price and revenue are long-tailed, so the code uses log transformations and winsorization for analysis while avoiding blind removal of valid luxury listings.
- Feature-quality analysis removes IDs, URLs, raw text, target-derived price columns, and high-cardinality memorization fields. It also checks redundant numeric blocks such as availability windows and review-count features.
- The modeling stage uses task-specific feature lists: the price model cannot use price-derived variables, and the superhost model cannot use superhost-derived columns.

How to interpret the Airbnb figures: the city row-count chart shows sampling imbalance; the missing-value chart shows why imputation must be justified; the skewed-variable histograms show why linear models struggle with raw prices; and the correlation heatmap reveals redundant feature blocks that can inflate dimensionality without adding new information.

Dataset 2 workflow - NYC 311 service requests

What the assignment wanted: NYC 311 required complaint-category classification and resolution-time regression. It specifically required intelligent sampling to maintain distribution integrity, duplicate report handling, and meaningful temporal features. The notebook treats the dataset as operational service-log data rather than as a clean classification table.

- The loader reads the large ZIP/CSV structure robustly and creates a manageable stratified sample rather than a naive random sample.
- Sampling integrity is checked by comparing complaint distributions before and after sampling. This is essential because rare complaint types can disappear if sampling is careless.
- Duplicate handling separates exact duplicates from near-duplicate service reports, because repeated reports can represent either duplicate data entry or genuine repeated citizen complaints.
- The regression target is `resolution_time_hours`, computed from `Created Date` and `Closed Date`. Invalid negative or missing resolution times are removed or flagged.
- Temporal features are engineered from service timestamps: hour, day of week, month, weekend, after-hours, season, and cyclic encodings where appropriate.
- Borough, channel, agency, location, and coordinate features are treated as operational context, but high-cardinality leakage-like identifiers are removed.

- Outlier analysis focuses on resolution time and service-volume patterns, but target-derived outlier flags are not allowed into final model inputs.
- The final analysis explains that NYC 311 is hard because the same complaint category can have very different resolution times depending on borough, agency, channel, and temporal workload.

How to interpret the NYC figures: the top-complaint chart explains label imbalance; the borough chart reveals geographic sampling bias; and the resolution-time chart demonstrates why regression is hard and why long-tailed service delay must be treated with robust methods.

Dataset 3 workflow - IBM HR Employee Attrition

What the assignment wanted: HR Attrition required root-driver investigation, rigorous feature selection, and imbalance strategies such as SMOTE and undersampling. The notebook therefore treats the data as a high-stakes human-resource classification task where accuracy alone is misleading.

- The target Attrition is converted into attrition_binary. The code then measures class imbalance, showing that employees who leave are the minority class.
- Because the dataset has almost no missing values, missingness discussion focuses on the absence of observed missingness rather than on artificial imputation.
- Constant and non-informative features such as EmployeeCount, Over18, and StandardHours are removed because they cannot help prediction.
- Domain features are engineered to express HR risk mechanisms: overtime risk, long tenure without promotion, tenure ratios, satisfaction flags, and promotion stagnation variables.
- Root-driver analysis uses group comparisons and statistical association logic to identify variables related to attrition, such as OverTime, JobSatisfaction, EnvironmentSatisfaction, BusinessTravel, DistanceFromHome, and tenure/promotion variables.
- Sampling strategies are explicitly compared: original data, undersampling, oversampling, and SMOTE-like synthetic minority sampling. This is necessary because a high-accuracy model can still miss most employees likely to leave.
- Model selection emphasizes recall, F1, and balanced accuracy rather than raw accuracy because the minority attrition class is the operationally important class.
- Fairness analysis checks group reliability across Gender, Department, JobRole, MaritalStatus, BusinessTravel, OverTime, and AgeGroup.

How to interpret the HR figures: the class-imbalance chart explains why accuracy alone is not enough; numeric distribution plots show which HR variables have natural operational ranges; and model-comparison tables show why SMOTE can reduce overall accuracy but substantially improve attrition recall.

Dataset 4 workflow - Online Retail transactions

What the assignment wanted: Online Retail required converting raw transaction logs into customer-level behavioral features before modeling. This notebook therefore does not model invoice rows directly. It first creates a temporal customer-history table and then predicts future spending and future segment.

- The Excel loader reads the Online Retail worksheet and inspects invoice-level columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country.
- Transaction cleaning removes or separates cancellations, negative quantities, missing CustomerID rows, invalid prices, and duplicate lines.
- The code creates a temporal split: historical transactions become features, while a later future window becomes the source of labels. This prevents using future information to predict itself.
- Customer-level aggregation creates RFM and behavior variables: recency, frequency, monetary value, unique products, purchase rate, average invoice value, customer age, country, and historical segment indicators.
- The regression target is future_spending. The classification target is future_customer_segment, derived from future behavior thresholds.
- Leakage checks are critical: future_spending, future_invoice_count, future_quantity, and other future columns are excluded from feature engineering and outlier-feature construction.

- Outlier analysis focuses on historical customer behavior, not future target values. This matters because high spenders are both valid business cases and possible sources of large regression error.
- Country-level bias analysis is included because the United Kingdom dominates the dataset and can hide weak performance on smaller countries.

How to interpret the Retail figures: missing-value plots reveal transaction-record incompleteness; customer outlier plots reveal the heavy-tailed distribution of spending and frequency; final model metrics show how difficult future spending is compared with segment classification; and country-level accuracy reveals geographic imbalance.

0.4 Written Analysis Logic Used Throughout the Report

The written analysis in the report follows a fixed logic for every chart and table. First, the report states what the figure measures. Second, it explains what problem the figure reveals. Third, it connects that problem to a preprocessing or modeling decision. Fourth, it explains how that decision affects the trustworthiness of the final model. This structure was used so that the report does not simply say a chart is high or low; it explains why the pattern matters for machine learning.

Evidence type	Question answered	Typical interpretation
Missing-value chart/table	Which columns are unreliable or structurally absent?	High missingness may require dropping, indicator engineering, or domain-specific imputation rather than a universal mean/median rule.
Distribution/histogram	Is the variable symmetric, skewed, long-tailed, or multimodal?	Skewed variables justify log transforms, robust scaling, winsorization, and caution with linear models.
Outlier comparison table	Do Z-score, IQR, and Isolation Forest agree?	Disagreement means some observations are only univariate extremes while others are unusual multi-feature profiles.
Correlation/redundancy heatmap	Which features carry duplicate information?	Highly correlated features can increase computation and instability without improving generalization.
Model comparison table	Which model family best matches the engineered data manifold?	Linear methods are interpretable; RBF kernels capture local nonlinear patterns; trees capture thresholds; KNN is sensitive to scaling and sparsity.
Failure-analysis table	Where did the model fail worst?	Worst predictions reveal whether errors come from noisy labels, rare cases, missing context, class imbalance, or model limitations.
Bias/fairness table	Does performance change by city, borough, department, or country?	Group-level performance differences indicate real-world deployment risk even when aggregate metrics look acceptable.
Computational analysis	Can the method scale to the full data?	Kernel methods can become impractical because kernel matrices require $O(n^2)$ memory and $O(n^3)$ operations for inversion/eigendecomposition.

0.5 Why the Same Modeling Code Was Not Enough

A central conclusion of the project is that the algorithm alone did not determine success. Airbnb and Online Retail are dominated by long-tailed monetary behavior; NYC 311 is dominated by operational delay, geography, and time; HR is dominated by class imbalance and ambiguous organizational labels. Therefore, the notebooks intentionally spend more code on cleaning, auditing, and feature construction than on model fitting. This is consistent with the grading emphasis in the assignment and with the reality of production machine learning.

For example, a kernel model may look powerful mathematically, but it cannot fix leakage, missing labels, duplicated records, wrong temporal splits, or a target that was created incorrectly. Similarly, a linear model may appear weak on raw data but become competitive after careful log transformation, robust scaling, and leakage-safe feature selection. The report therefore evaluates models only after showing how the modeling dataset was constructed.

0.6 How to Read the Rest of the Report

The rest of the report is organized in the same order as the four notebooks and the assignment. For each dataset, the report first describes the raw topology and task definition, then explains data cleaning and feature engineering, then interprets figures and tables, then discusses modeling results, kernel behavior, failure modes, and bias risks. The cross-dataset section at the end

synthesizes which dataset was hardest to clean, which needed the most feature engineering, which benefited most from kernels, and why textbook datasets underestimate these issues.

1. Assignment Interpretation and Required Deliverables

The assignment asks for a mini-conference style report, not a raw notebook. It requires four datasets, each with a different data topology and operational risk: Airbnb listings, NYC 311 service requests, HR attrition records, and Online Retail transactions. Each dataset must be cleaned with a bespoke pipeline rather than a universal preprocessing recipe.

Assignment compliance map

Assignment component	What was required	How this report addresses it
Data quality investigation	Missing data mechanism, imputation math, outliers, leakage, redundant features, label quality	Each dataset section has dedicated data-quality subsections and figures
Algorithmic implementation	Linear/logistic regression, KNN, decision tree, KRR, kernel SVM, kernel KNN, KPCA from scratch	Model sections summarize from-scratch implementations and metrics where executed
Kernel investigation	Explain why kernels work/fail on each dataset, not only quote metrics	Kernel interpretation is dataset-specific and linked to topology
Research-level discussion	Cross-dataset comparison, failure analysis, bias/fairness, computation, theory vs reality	A final synthesis compares all four manifolds
Final report format	20-35 page mini-conference paper plus notebooks/scripts appendix	This document is structured as a full report and includes figure/table evidence

2. Dataset Description and Topology

The four datasets are not merely four tables. They represent four different real-world manifolds: spatial marketplace data, temporal service-request data, organizational HR records, and transactional customer behavior. The table below summarizes the learning tasks and main engineering challenge for each dataset.

Four dataset topology summary

Dataset	Prediction task(s)	Topology	Main risk
1. Airbnb Open Data	Price regression and superhost classification	Geographic + property + host behavior manifold	Missingness, price outliers, city heterogeneity, leakage through target-derived variables
2. NYC 311 Service Requests	Complaint category classification and resolution-time regression	Temporal + geographic + operational complaint manifold	Huge data volume, duplicates, temporal drift, borough/channel bias
3. IBM HR Attrition	Employee attrition classification	Small structured HR/personnel manifold	Severe class imbalance, ambiguous labels, historical HR bias
4. Online Retail Transactions	Future spending regression and customer segment classification	Transaction logs aggregated into customer behavior manifold	Leakage from future behavior, skewed monetary variables, country concentration

3. Dataset 1 - Airbnb Five-City Open Data

The first notebook generalizes the original single-city Airbnb pipeline to five cities: Amsterdam, Barcelona, Chicago, New York, and Madrid. The assignment required price regression and superhost classification, with special emphasis on missing-value imputation, feature-dropping criteria, and outlier handling. The five-city setting makes the problem more realistic because each city has different listing volume, price scale, missingness rate, and superhost distribution.

Airbnb city-level diagnostic table

city	rows	columns	price_nonmissing	price_missing_rate	price_median	price_mean	price_p95	price_max	superhost_nonmissing	superhost_rate
Amsterdam	10480	80	5874	0.4395	222.0000	336.7852	550.0000	80,018.00	10365	0.1796
Barcelona	16107	91	12730	0.2097	176.9200	232.1052	525.0000	10,542.22	15982	0.2414
Chicago	8660	80	7825	0.0964	153.0000	601.3048	587.8000	50,032.00	8371	0.4655
New York	30259	91	21515	0.2890	174.6900	278.2295	759.0000	30,972.96	29910	0.2372
Madrid	23002	91	17693	0.2308	133.5000	179.2767	405.6720	36,883.00	22749	0.2279

The table shows that the dataset is not homogeneous. Amsterdam has the highest price missingness, Chicago has the highest superhost rate, and New York has the largest number of rows. The median price also changes by city, which means that a global model must learn city effects or include city-level encoding. Using one city only would hide this domain shift.

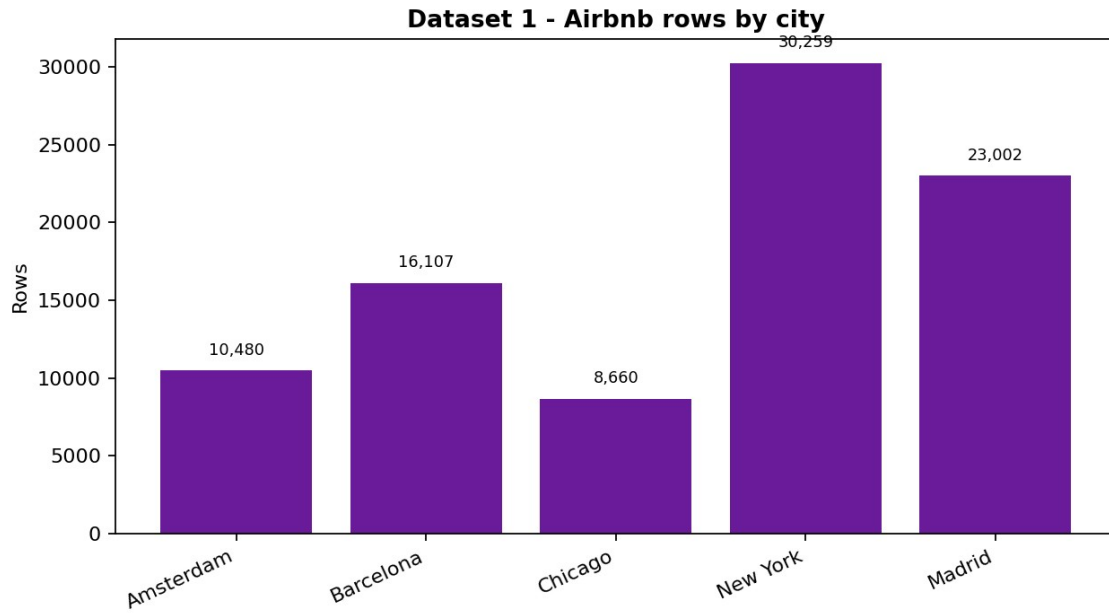


Figure 1.1. Airbnb row count by city.

New York and Madrid dominate the row count, so without city-aware evaluation the model can be biased toward large-city patterns.

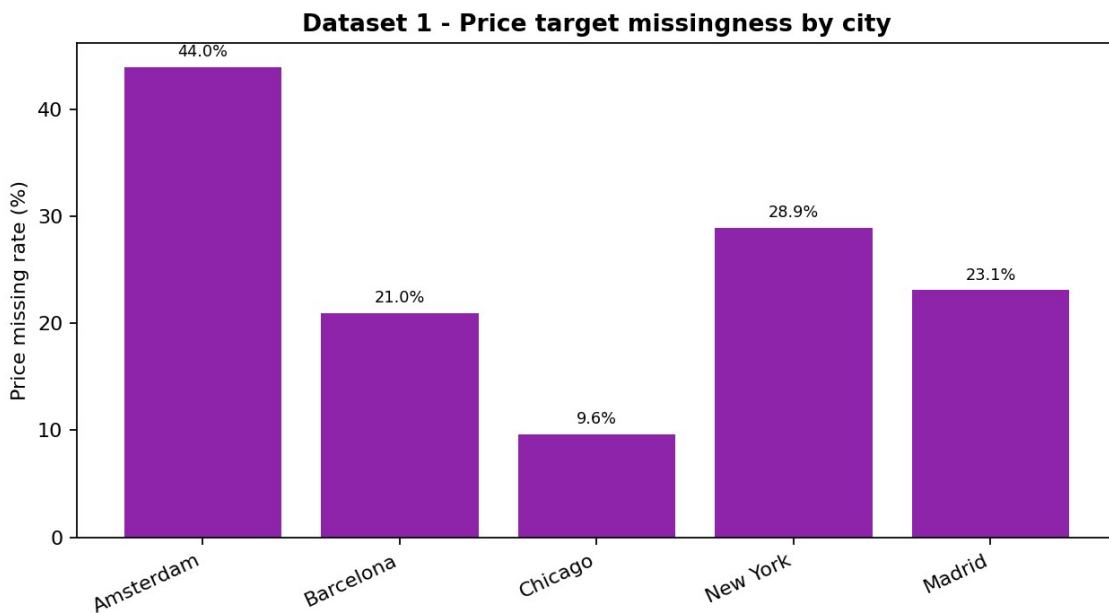


Figure 1.2. Airbnb price missingness by city.

Missingness is not uniform across cities. Amsterdam has a much higher price-missing rate than Chicago, so missingness is not simply random noise.

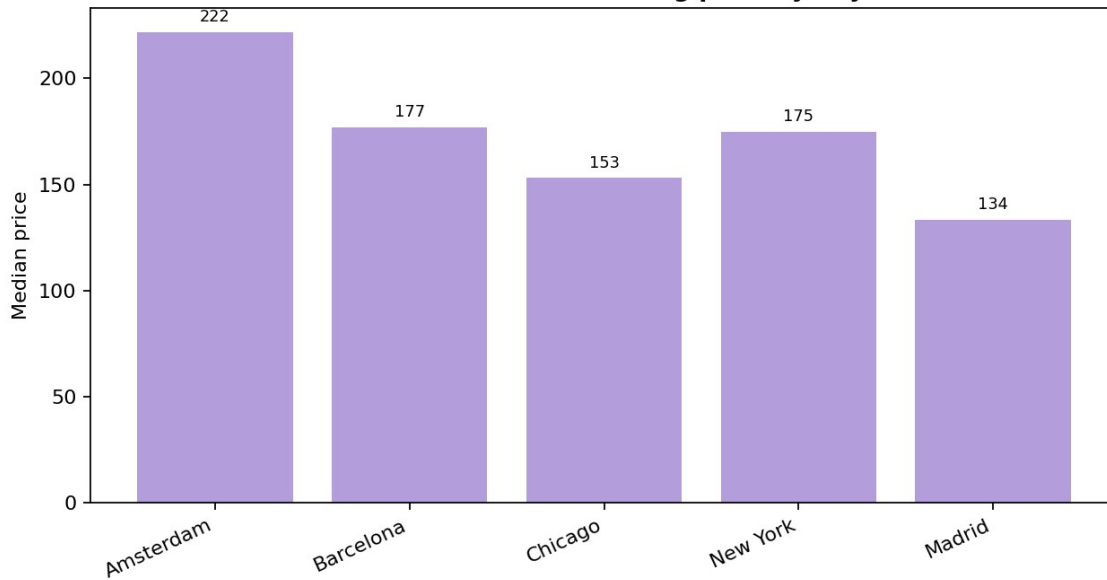
Dataset 1 - Median listing price by city

Figure 1.3. Airbnb median price by city.

The city median price chart shows that city identity is a strong structural variable. A single global median imputation would be too crude.

Airbnb data-quality workflow

The notebook first loads all city CSV files, adds a city column, unions non-identical schemas, and then standardizes the targets. Price strings are converted to numeric values and host_is_superhost is converted to a binary target. Missingness is investigated with missingness indicators, Mann-Whitney/Chi-square association tests, and domain rules. Raw text fields such as description and host_about are not directly imputed; instead, the pipeline engineers binary has-text indicators and text-length features.

Airbnb missingness classification - strongest columns

column	missing_count	missing_rate	number_of_significant_associations	missingness_mechanism
neighborhood_overview	5288	0.5046	20	MAR / Suspected MNAR
host_about	4970	0.4742	20	MAR / Suspected MNAR
price	4606	0.4395	20	MAR / Suspected MNAR
price_numeric	4606	0.4395	20	MAR / Suspected MNAR
estimated_revenue_l365d	4606	0.4395	20	MAR / Suspected MNAR
beds	4576	0.4366	20	MAR
bathrooms	4548	0.4340	21	MAR
host_response_time	3797	0.3623	22	MAR
host_response_rate	3797	0.3623	22	MAR
host_acceptance_rate	2430	0.2319	23	MAR
host_location	1232	0.1176	20	MAR
review_scores_cleanliness	1098	0.1048	19	MAR
first_review	1097	0.1047	19	MAR
last_review	1097	0.1047	19	MAR
review_scores_rating	1097	0.1047	19	MAR

Airbnb imputation strategy examples

column	missing_rate	imputation_strategy	reason
host_neighbourhood	0.7338	Unknown category	Missing category may be informative; using Unk...
neighborhood_overview	0.5046	text indicator + text length	Raw text cannot be meaningfully imputed by mea...
neighbourhood	0.5046	Unknown category	Missing category may be informative; using Unk...
host_about	0.4742	text indicator + text length	Raw text cannot be meaningfully imputed by mea...
estimated_revenue_l365d	0.4395	median	Median is robust to skewness and outliers in r...
beds	0.4366	median	Median is robust to skewness and outliers in r...
bathrooms	0.4340	median	Median is robust to skewness and outliers in r...
host_response_rate	0.3623	Unknown category	Missing category may be informative; using Unk...
host_response_time	0.3623	Unknown category	Missing category may be informative; using Unk...
host_acceptance_rate	0.2319	Unknown category	Missing category may be informative; using Unk...
host_location	0.1176	Unknown category	Missing category may be informative; using Unk...
review_scores_cleanliness	0.1048	median	Median is robust to skewness and outliers in r...
first_review	0.1047	Unknown category	Missing category may be informative; using Unk...
last_review	0.1047	Unknown category	Missing category may be informative; using Unk...

review_scores_accuracy	0.1047	median	Median is robust to skewness and outliers in r...
review_scores_checkin	0.1047	median	Median is robust to skewness and outliers in r...
review_scores_communication	0.1047	median	Median is robust to skewness and outliers in r...
review_scores_location	0.1047	median	Median is robust to skewness and outliers in r...

Airbnb outlier analysis and feature-quality results

The Airbnb price and revenue variables are heavily right-skewed. The notebook compares Z-score, IQR, and a Simple Isolation Forest implemented from scratch with NumPy. The final strategy does not blindly delete all outliers. Instead, it keeps diagnostic flags for investigation, applies log transformations to skewed variables, uses winsorized alternatives, and reserves only invalid values for removal.

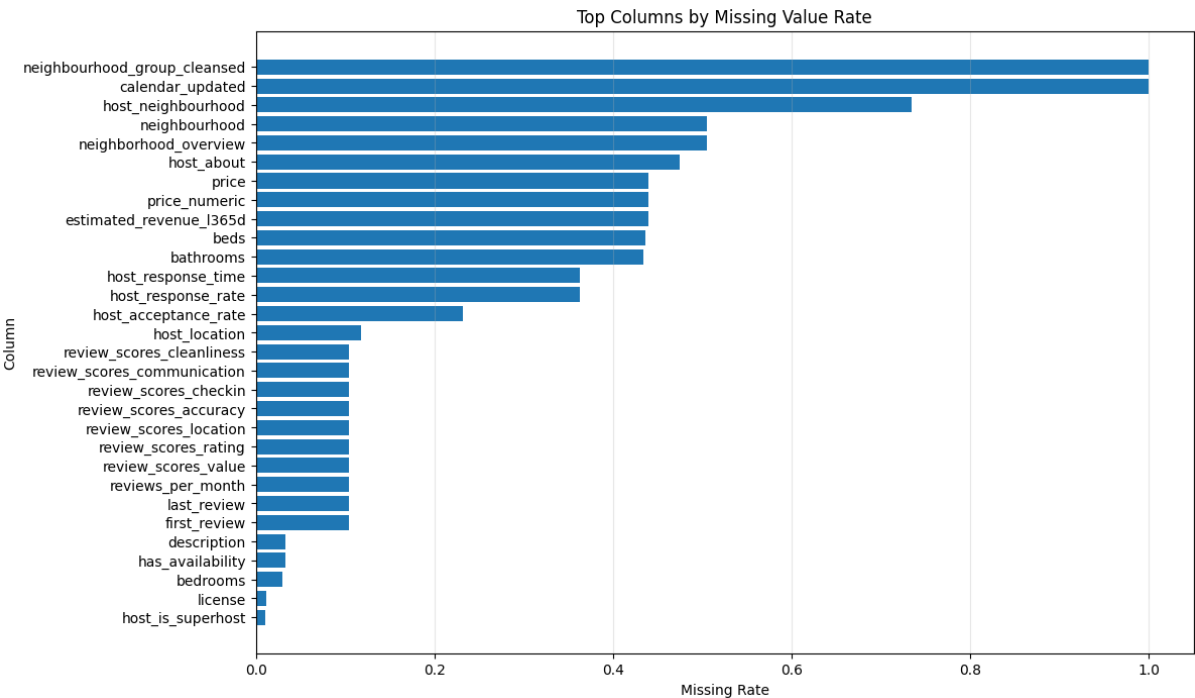
Airbnb outlier method comparison

Unnamed: 0	method	outlier_count	outlier_rate
0	Z-score	1025	0.0978
1	IQR	4835	0.4614
2	Simple Isolation Forest	524	0.0500

Airbnb top redundant numeric feature pairs

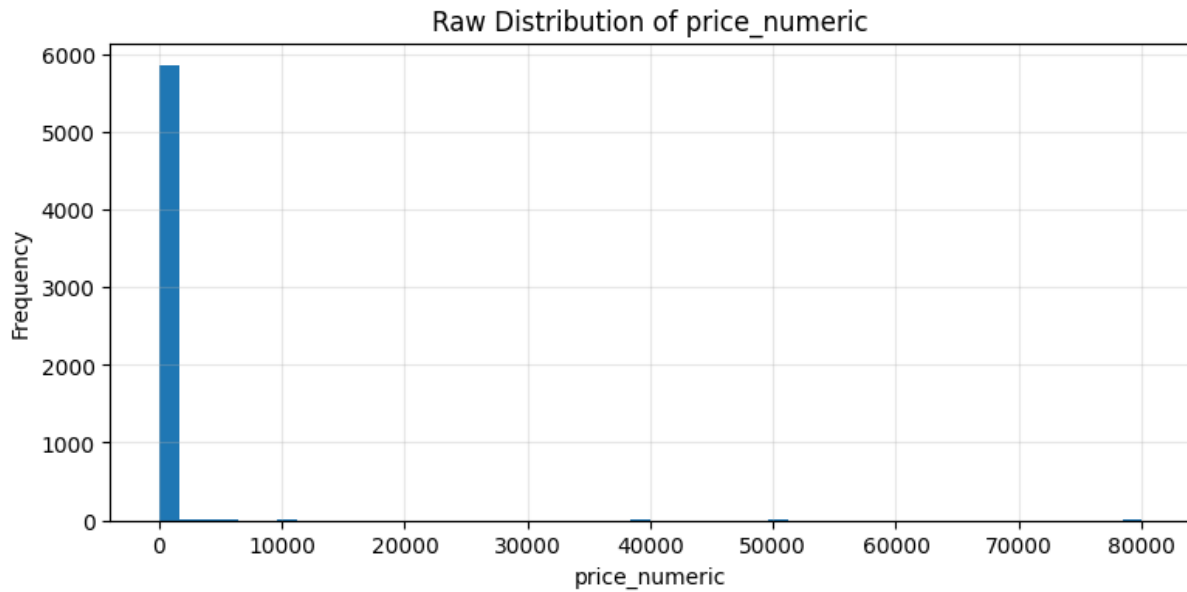
Unnamed: 0	feature_1	feature_2	abs_spearman_corr
0	minimum_nights	log1p_minimum_nights	1.0000
1	maximum_nights	log1p_maximum_nights	1.0000
2	number_of_reviews	log1p_number_of_reviews	1.0000
3	estimated_occupancy_l365d	estimated_occupancy_l365d_winsorized_p01_p99	1.0000
4	has_description	is_missing_description	1.0000
5	has_neighborhood_overview	is_missing_neighborhood_overview	1.0000
6	is_missing_price	is_missing_price_numeric	1.0000
7	has_host_about	is_missing_host_about	1.0000
8	is_missing_price_numeric	is_missing_estimated_revenue_l365d	1.0000
9	is_missing_host_is_superhost	is_missing_superhost_binary	1.0000
10	is_missing_host_response_time	is_missing_host_response_rate	1.0000
11	is_missing_price	is_missing_estimated_revenue_l365d	1.0000
12	estimated_revenue_l365d	log1p_estimated_revenue_l365d	1.0000
13	reviews_per_month	log1p_reviews_per_month	1.0000
14	estimated_occupancy_l365d	log1p_estimated_occupancy_l365d	1.0000
15	price_numeric	log1p_price_numeric	1.0000
16	is_missing_review_scores_location	is_missing_review_scores_value	1.0000
17	is_missing_review_scores_communication	is_missing_reviews_per_month	1.0000
18	is_missing_review_scores_communication	is_missing_review_scores_location	1.0000
19	is_missing_review_scores_communication	is_missing_review_scores_value	1.0000

Airbnb notebook figures and detailed interpretation



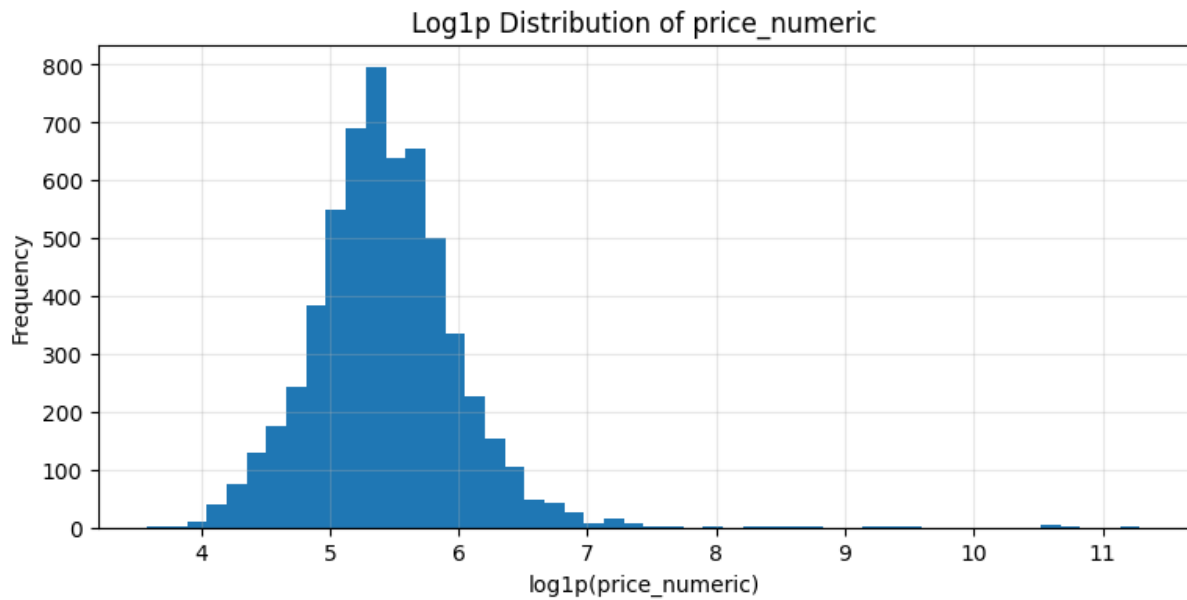
Airbnb Figure 1. airbnb 01 cell19 6 Visualizing Missing Value Rates

The missing-value chart identifies which columns require dropping, imputation, or indicator engineering. It supports the written imputation strategy.



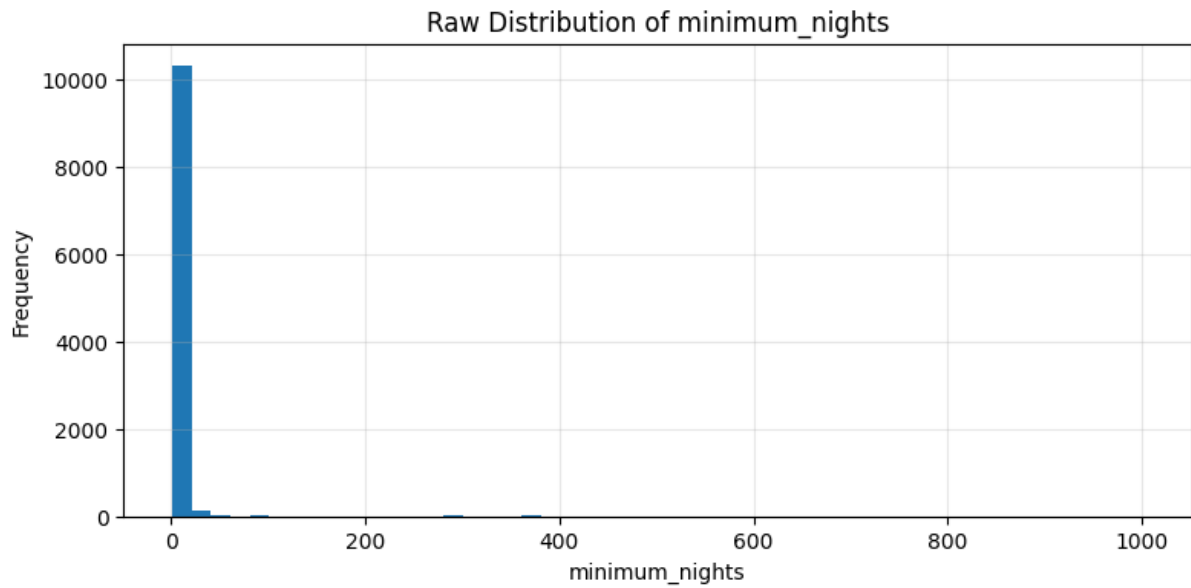
Airbnb Figure 2. airbnb 02 cell53 24 Visualizing Important Skewed Variables

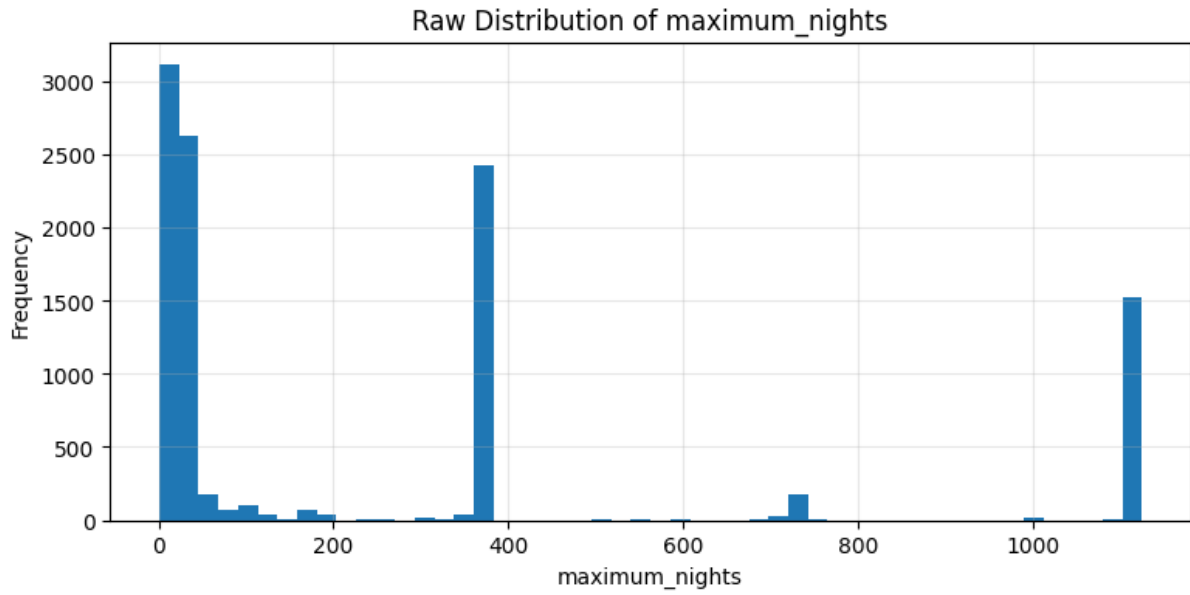
Raw price is extremely right-skewed, showing that ordinary least squares on raw price would be dominated by luxury and erroneous high-price listings.



Airbnb Figure 3. airbnb 03 cell53 24 Visualizing Important Skewed Variables

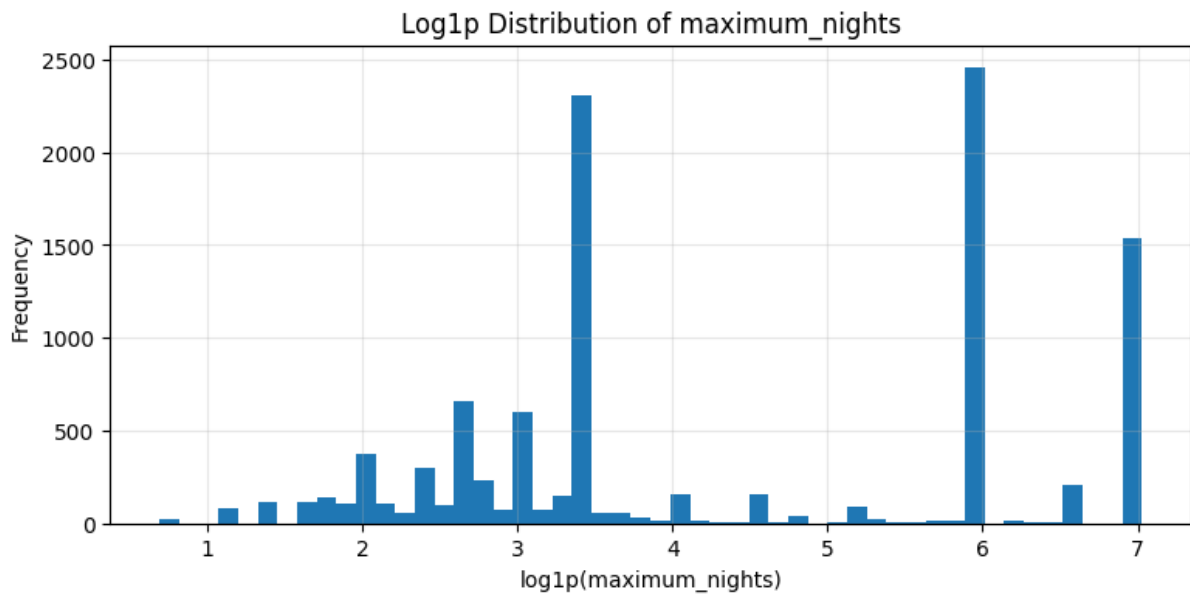
The log price plot compresses the long tail and makes the main body of listing prices more learnable for linear and kernel models.





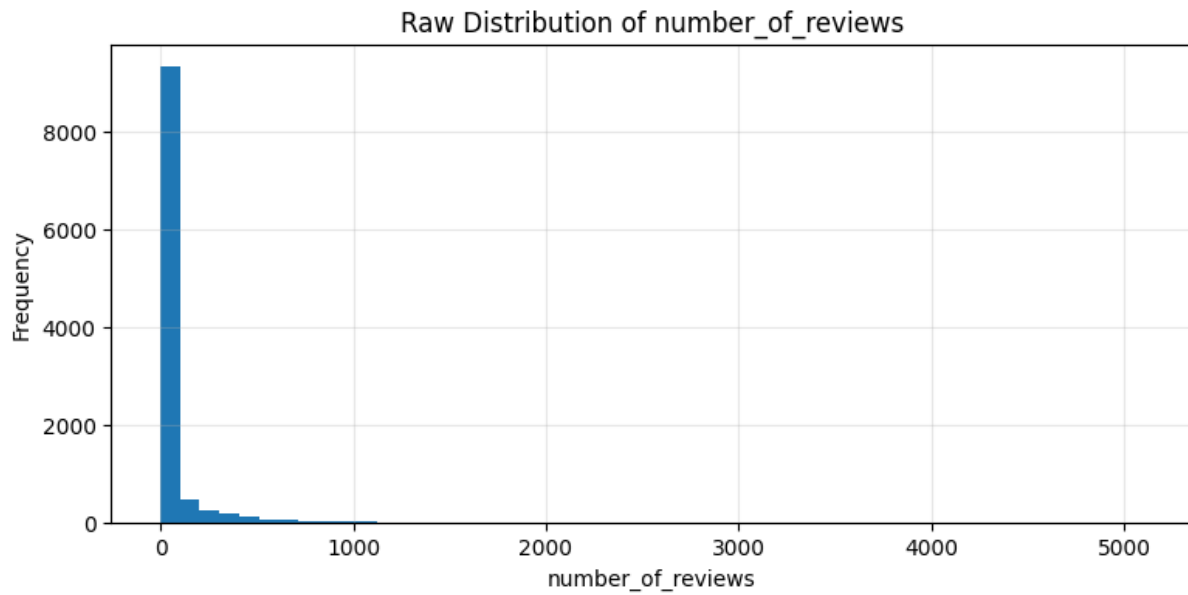
Airbnb Figure 6. airbnb 06 cell53 24 Visualizing Important Skewed Variables

Raw maximum_nights is also skewed and can encode default platform values or long-stay policies rather than true demand.



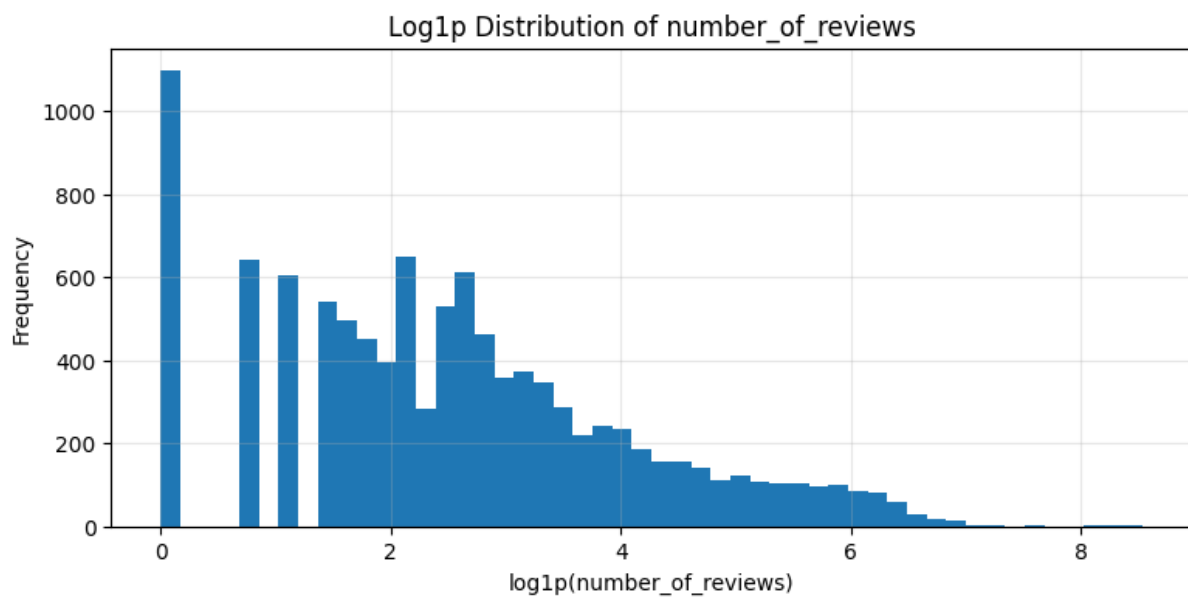
Airbnb Figure 7. airbnb 07 cell53 24 Visualizing Important Skewed Variables

The log maximum-nights chart makes the distribution easier to compare across cities and supports robust scaling.



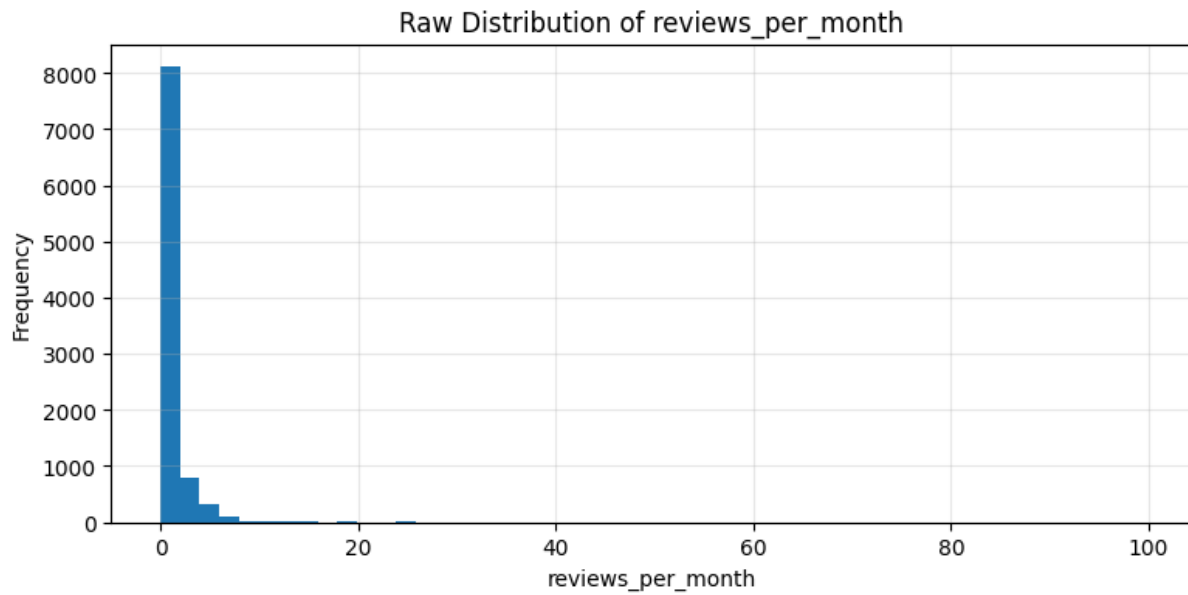
Airbnb Figure 8. airbnb 08 cell53 24 Visualizing Important Skewed Variables

Review counts are long-tailed because a few old/popular listings accumulate far more reviews than typical listings.



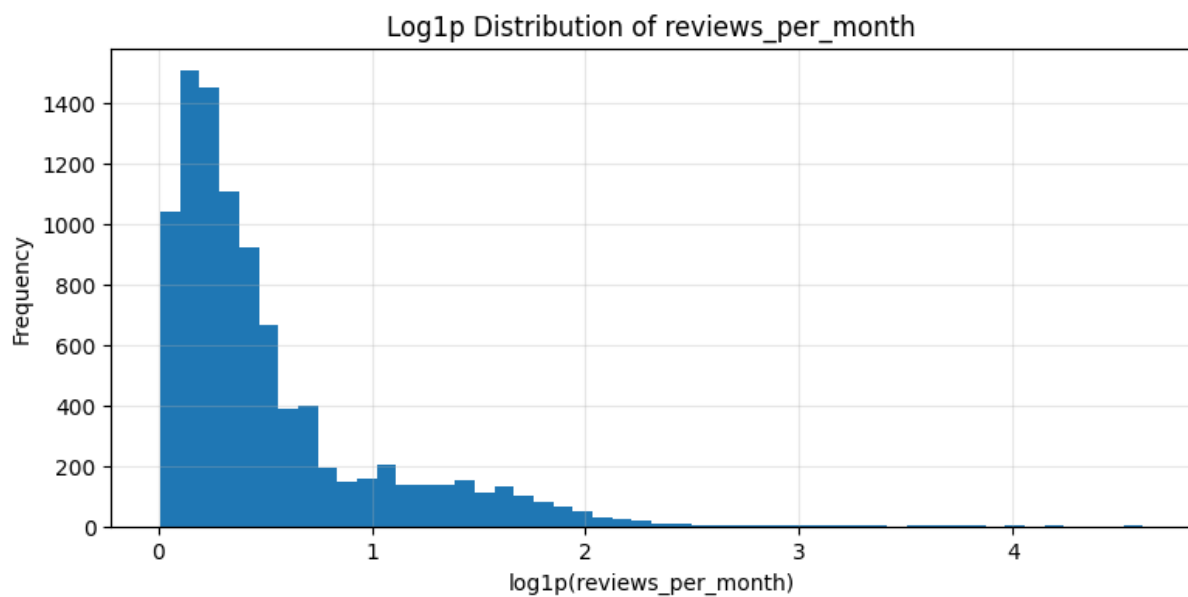
Airbnb Figure 9. airbnb 09 cell53 24 Visualizing Important Skewed Variables

The log review-count distribution is more suitable for distance-based models such as KNN and kernel KNN.



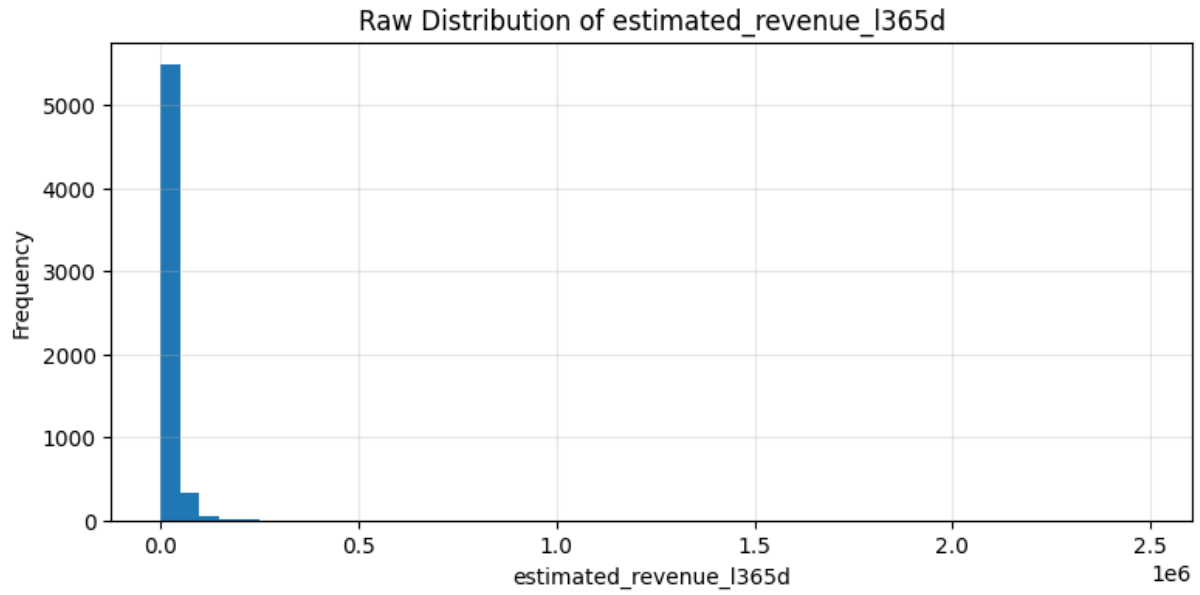
Airbnb Figure 10. airbnb 10 cell53 24 Visualizing Important Skewed Variables

reviews_per_month is operationally meaningful but skewed; it captures activity intensity rather than static property quality.



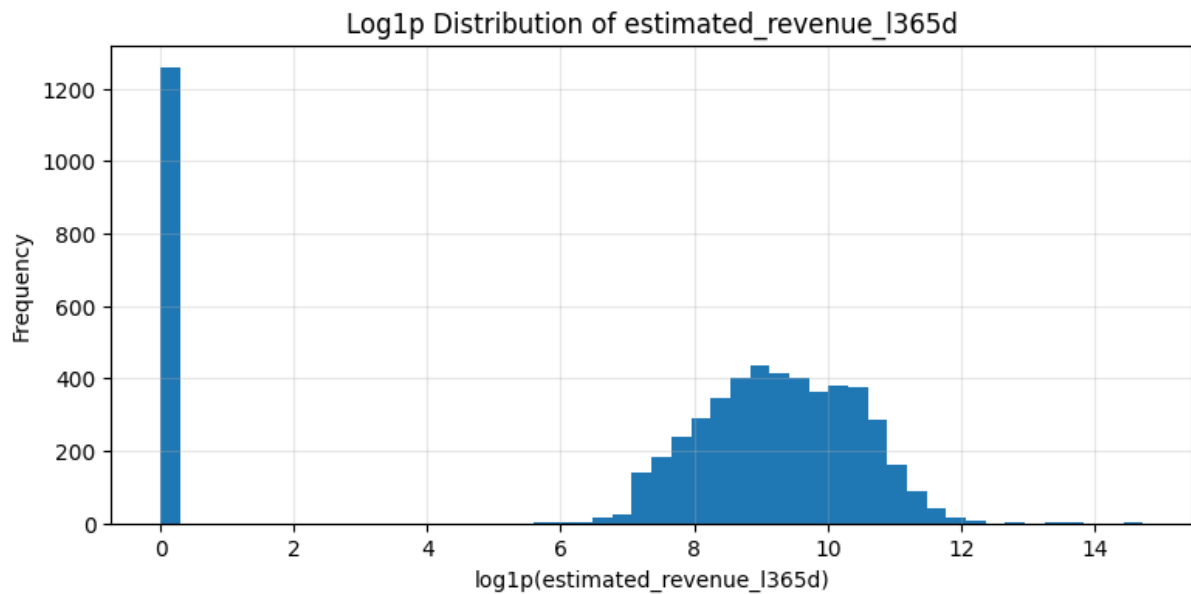
Airbnb Figure 11. airbnb 11 cell53 24 Visualizing Important Skewed Variables

The log reviews-per-month plot reduces the effect of very active listings and supports robust regression/classification.



Airbnb Figure 12. airbnb 12 cell53 24 Visualizing Important Skewed Variables

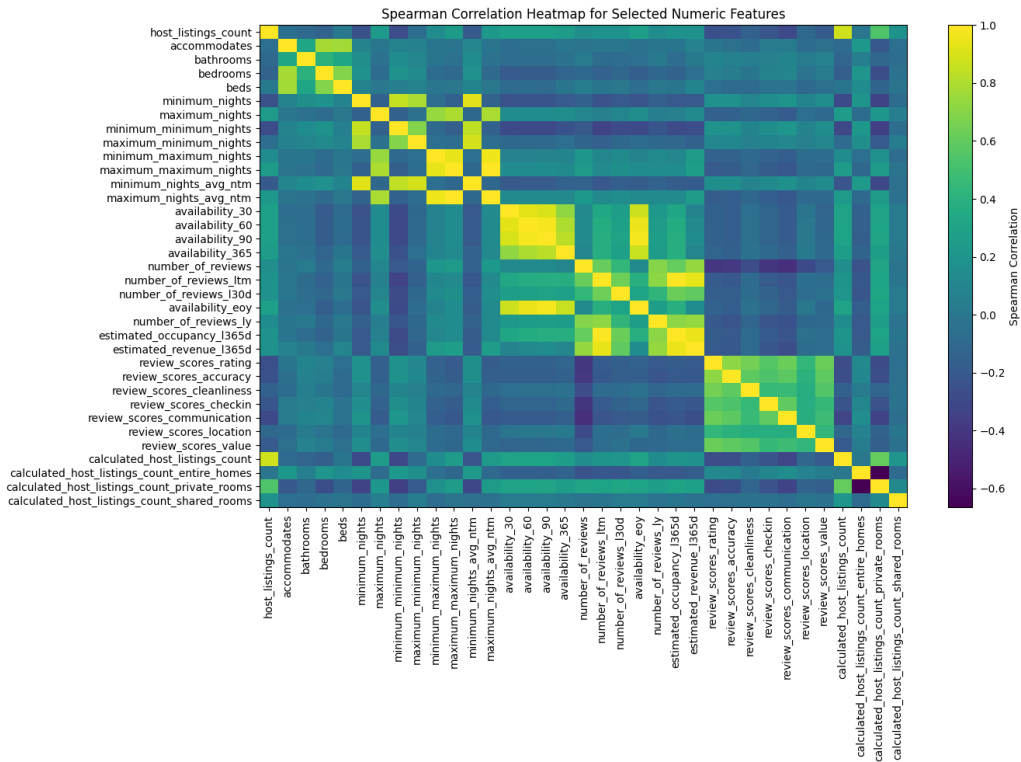
Estimated revenue is highly right-skewed and also high-risk for leakage in price prediction if mechanically derived from price.



Airbnb Figure 13. airbnb 13 cell53 24 Visualizing Important Skewed Variables

The log revenue chart shows why revenue variables must be handled carefully and often excluded from price prediction inputs.

ML Homework 3 - Expanded Four-Dataset Report



Airbnb Figure 14. airbnb 14 cell93 44 Numeric Correlation Heatmap

The correlation heatmap reveals redundant feature blocks, especially availability windows, review variables, and host listing counts.

Airbnb modeling and interpretation

The implemented Airbnb notebook contains the complete modeling pipeline through non-kernel and kernel methods, including from-scratch Linear Regression, Logistic Regression, KNN, Decision Tree, Kernel Ridge Regression, Kernel KNN, Kernel SVM, and KPCA. The available executed summary indicates 65,637 price-observed listings, a global median price of 163.00, a median-baseline MAE of 190.68, and a superhost majority-class baseline accuracy of 0.749. The baseline itself is important: it shows that Airbnb has enough skew and city heterogeneity that good models must outperform a simple median or majority-class strategy, not merely produce reasonable-looking predictions.

Code workflow walkthrough - Dataset 1 - Airbnb Five-City

This table explains the notebook workflow section by section. It is included because the assignment grades the reasoning process and data engineering pipeline, not only final metrics. The notebook contains 92 documented sections.

Section	Notebook section title	What it does
1	Homework 3 — Kernel and Non-Kernel Learning Under Real-World Data Imperfections	Runs kernel experiments and explains why each kernel works or fails on the dataset.
2	1. Loading the Five-City Airbnb Dataset	Loads the raw dataset and verifies that files, columns, and row counts are correct.
3	1A. City-Level Coverage Audit	Supports the dataset-specific pipeline and documents an intermediate result.
4	2. Basic Schema Inspection	Inspects dtypes, missing rates, unique values, and raw structure before cleaning.
5	3. Target Identification for Airbnb Tasks	Defines and validates supervised learning targets while preventing target leakage.
6	4. Cleaning the Target Variables	Defines and validates supervised learning targets while preventing target leakage.
7	4A. City-Level Target Diagnostics	Defines and validates supervised learning targets while preventing target leakage.
8	5. Initial Missing Value Analysis	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
9	5A. City-Level Missing Value Analysis	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
10	6. Visualizing Missing Value Rates	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
11	7. First Column Decision Table	Supports the dataset-specific pipeline and documents an intermediate result.

ML Homework 3 - Expanded Four-Dataset Report

12	8. Engineering Binary Indicators from Descriptive Text Features	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
13	9. Duplicate Rows and Near-Constant Features	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
14	10. Save the Initial Working Dataframe	Supports the dataset-specific pipeline and documents an intermediate result.
15	11. Missingness Mechanisms: MCAR, MAR, and MNAR	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
16	12. Selecting Important Columns for Missingness Investigation	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
17	13. Creating Missingness Indicator Variables	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
18	15. Summarizing Strong Missingness Associations	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
19	16. Evidence-Based Missingness Classification	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
20	17. Designing the Imputation Strategy	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
21	18. Applying Initial Imputation	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
22	19. Saving Missingness and Imputation Outputs	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
23	20. Report-Ready Missingness Summary	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
24	21. Outlier Analysis Overview	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
25	22. Selecting Numeric Variables for Outlier Analysis	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
26	24. Visualizing Important Skewed Variables	Supports the dataset-specific pipeline and documents an intermediate result.
27	25. Z-score Outlier Detection	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
28	26. IQR Outlier Detection	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
29	27. Simple Isolation Forest from Scratch	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
30	28. Preparing Data for Simple Isolation Forest	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
31	29. Running the Simple Isolation Forest	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
32	30. Comparing Outlier Detection Methods	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
33	30A. City-Level Outlier Comparison	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
34	31. Inspecting the Most Suspicious Listings	Supports the dataset-specific pipeline and documents an intermediate result.
35	32. Outlier Treatment Strategy	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
36	33. Invalid Value Checks	Supports the dataset-specific pipeline and documents an intermediate result.
37	34. Final Outlier Method Comparison Summary	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
38	35. Saving Outlier Analysis Outputs	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
39	37. Feature Quality Analysis Overview	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
40	38. Loading the Working Dataframe	Loads the raw dataset and verifies that files, columns, and row counts are correct.
41	39. Reconfirming Target Variables	Defines and validates supervised learning targets while preventing target leakage.
42	40. Feature Type Inventory	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
43	41. Near-Constant Feature Detection	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
44	42. High-Cardinality Categorical Feature Detection	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
45	43. Numeric Redundancy Analysis Using Correlation	Supports the dataset-specific pipeline and documents an intermediate result.
46	44. Numeric Correlation Heatmap	Supports the dataset-specific pipeline and documents an intermediate result.
47	45. Data Leakage Candidate Detection	Supports the dataset-specific pipeline and documents an intermediate result.
48	46. Target Association Diagnostic	Defines and validates supervised learning targets while preventing target leakage.
49	47. Building the Final Feature Quality Decision Table	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
50	48. Creating Task-Specific Candidate Feature Lists	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
51	49. Saving Feature Quality Outputs	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
52	50. Report-Ready Feature Quality Summary	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
53	51. Modeling Dataset Construction Overview	Implements/evaluates from-scratch non-kernel and kernel learning models.
54	52. Loading Feature Quality Outputs and Working Dataframe	Loads the raw dataset and verifies that files, columns, and row counts are correct.
55	53. Final Target Validation	Defines and validates supervised learning targets while preventing target leakage.
56	54. Date Feature Engineering	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
57	55. Final Task-Specific Feature Lists	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.

Purple report - complete dataset-by-dataset analysis

ML Homework 3 - Expanded Four-Dataset Report

58	56. Train/Test Split from Scratch	Builds train/test matrices with from-scratch preprocessing.
59	57. From-Scratch Tabular Preprocessor	Builds train/test matrices with from-scratch preprocessing.
60	58. Building Final Price Regression Matrices	Supports the dataset-specific pipeline and documents an intermediate result.
61	59. Building Final Superhost Classification Matrices	Supports the dataset-specific pipeline and documents an intermediate result.
62	60. Baseline Dataset Sanity Checks	Supports the dataset-specific pipeline and documents an intermediate result.
63	61. Saving Final Modeling Datasets	Implements/evaluates from-scratch non-kernel and kernel learning models.
64	62. Report-Ready Dataset Construction Summary	Creates report-ready outputs, final audit checks, and deliverable indexes.
65	63. Evaluation Metrics from Scratch	Supports the dataset-specific pipeline and documents an intermediate result.
66	64. Experiment Utilities and Sampling Controls	Supports the dataset-specific pipeline and documents an intermediate result.
67	65. Linear Regression from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
68	66. Run Linear Regression for Price Prediction	Implements/evaluates from-scratch non-kernel and kernel learning models.
69	67. Logistic Regression from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
70	68. Run Logistic Regression for Superhost Classification	Implements/evaluates from-scratch non-kernel and kernel learning models.
71	69. KNN from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
72	70. Run KNN for Price Regression and Superhost Classification	Implements/evaluates from-scratch non-kernel and kernel learning models.
73	71. Decision Tree from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
74	72. Run Decision Tree Models	Implements/evaluates from-scratch non-kernel and kernel learning models.
75	73. Non-Kernel Results Summary	Runs kernel experiments and explains why each kernel works or fails on the dataset.
76	74. Kernel Investigation Setup	Runs kernel experiments and explains why each kernel works or fails on the dataset.
77	75. Kernel Functions from Scratch	Runs kernel experiments and explains why each kernel works or fails on the dataset.
78	76. Kernel Ridge Regression from Scratch	Runs kernel experiments and explains why each kernel works or fails on the dataset.
79	77. Run Kernel Ridge Regression Kernel Comparison	Runs kernel experiments and explains why each kernel works or fails on the dataset.
80	78. Kernel KNN from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
81	79. Run Kernel KNN for Regression and Classification	Implements/evaluates from-scratch non-kernel and kernel learning models.
82	80. Kernel SVM from Scratch Using Simplified SMO	Runs kernel experiments and explains why each kernel works or fails on the dataset.
83	81. Run Kernel SVM Kernel Comparison	Runs kernel experiments and explains why each kernel works or fails on the dataset.
84	82. Kernel PCA from Scratch	Runs kernel experiments and explains why each kernel works or fails on the dataset.
85	83. KPCA + Downstream Classifier	Runs kernel experiments and explains why each kernel works or fails on the dataset.
86	84. Kernel Results Summary and Dataset-Specific Interpretation	Runs kernel experiments and explains why each kernel works or fails on the dataset.
87	85. Failure Analysis: Worst Predictions	Audits the worst predictions and distinguishes model limitation from data-quality artifacts.
88	86. City-Level Bias, Fairness, and Geographic Reliability	Analyzes group-level reliability, sampling bias, temporal drift, and geographic/customer bias.
89	87. Computational Analysis	Discusses runtime, memory, and kernel matrix $O(n^2)/O(n^3)$ limits.
90	88. Theory vs. Reality Discussion Evidence	Supports the dataset-specific pipeline and documents an intermediate result.
91	89. Saving Final Results and Report Tables	Creates report-ready outputs, final audit checks, and deliverable indexes.
92	90. Report-Ready Final Airbnb Dataset Discussion	Creates report-ready outputs, final audit checks, and deliverable indexes.

4. Dataset 2 - NYC 311 Service Requests

The NYC 311 dataset is a large operational service-request dataset. The assignment required complaint-category classification, resolution-time regression, intelligent sampling, duplicate handling, and temporal feature engineering. Because the raw data is very large, the notebook uses chunked reading and stratified sampling to preserve distribution integrity while keeping kernel computations feasible.

NYC 311 top complaint categories

Complaint category	Count
Illegal Parking	114730
Noise - Residential	68814
Noise - Street/Sidewalk	37968
Blocked Driveway	33152
Street Condition	24761

ML Homework 3 - Expanded Four-Dataset Report

UNSANITARY CONDITION	22501
HEAT/HOT WATER	22073
Water System	15007
Abandoned Vehicle	14276
PLUMBING	13112
Noise - Commercial	12713
Noise	11841
Dirty Condition	11666
Noise - Vehicle	11311
PAINT/PLASTER	11002
Damaged Tree	10730
Encampment	9611
DOOR/WINDOW	8422
General Construction/Plumbing	8059
Traffic Signal Condition	8021

NYC 311 borough distribution

Borough	Count
BROOKLYN	212228
QUEENS	170819
MANHATTAN	132241
BRONX	125093
STATEN ISLAND	26228
Unspecified	840
__MISSING__	5

NYC 311 submission channel distribution

Channel	Count
ONLINE	302467
PHONE	164361
MOBILE	140466
UNKNOWN	60154
__MISSING__	5
OTHER	1

The processed NYC profile contains 667,454 records in the profiling summary. Illegal Parking is the largest category, but its majority baseline accuracy is only 0.169 on the sampled classification set. This means complaint-category classification is genuinely multi-class and cannot be solved by a trivial majority predictor. Resolution-time regression is also difficult: the median resolution time is 2.23 hours, while the 95th percentile is 70.85 hours and the 99th percentile is 122.92 hours.

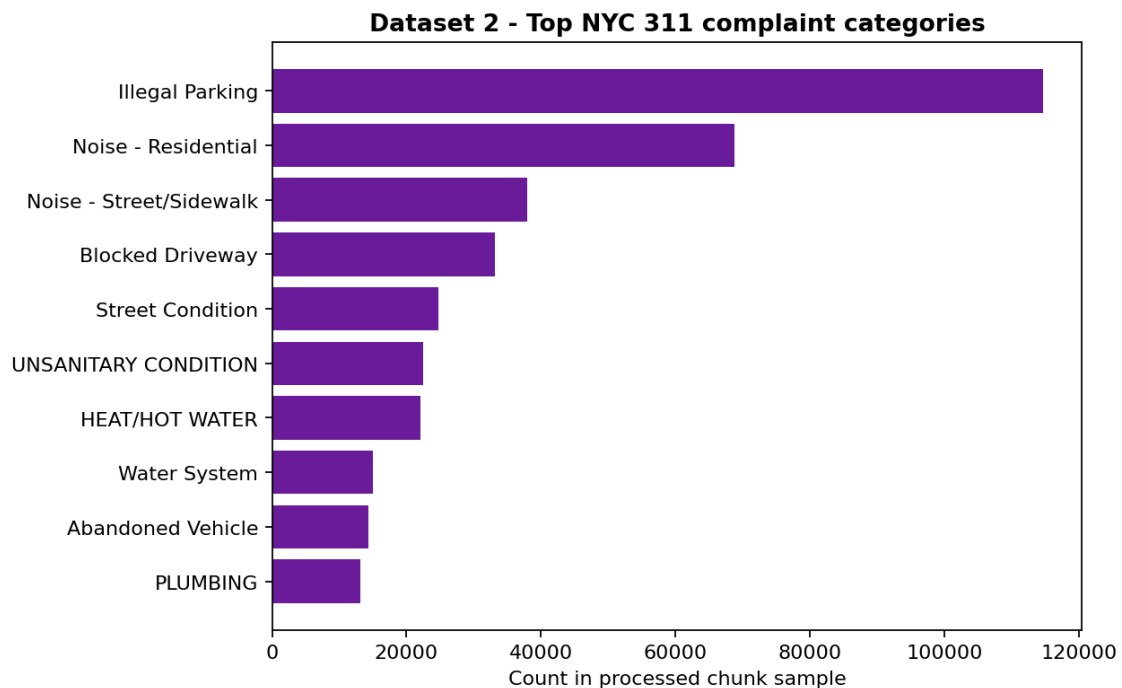


Figure 2.1. Top NYC 311 complaint categories.

The complaint distribution is highly imbalanced, but not dominated by a single class. This justifies stratified sampling and macro-averaged evaluation.

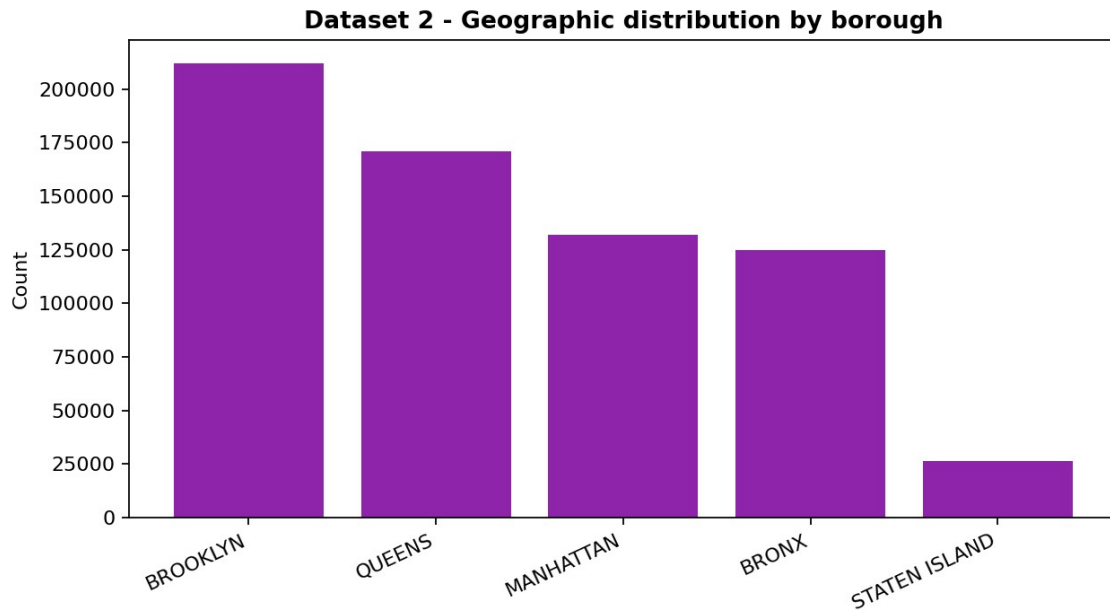


Figure 2.2. NYC 311 borough distribution.

Brooklyn, Queens, Manhattan, and Bronx dominate the sample. Borough-level imbalance means geographic bias and service-capacity differences must be discussed.

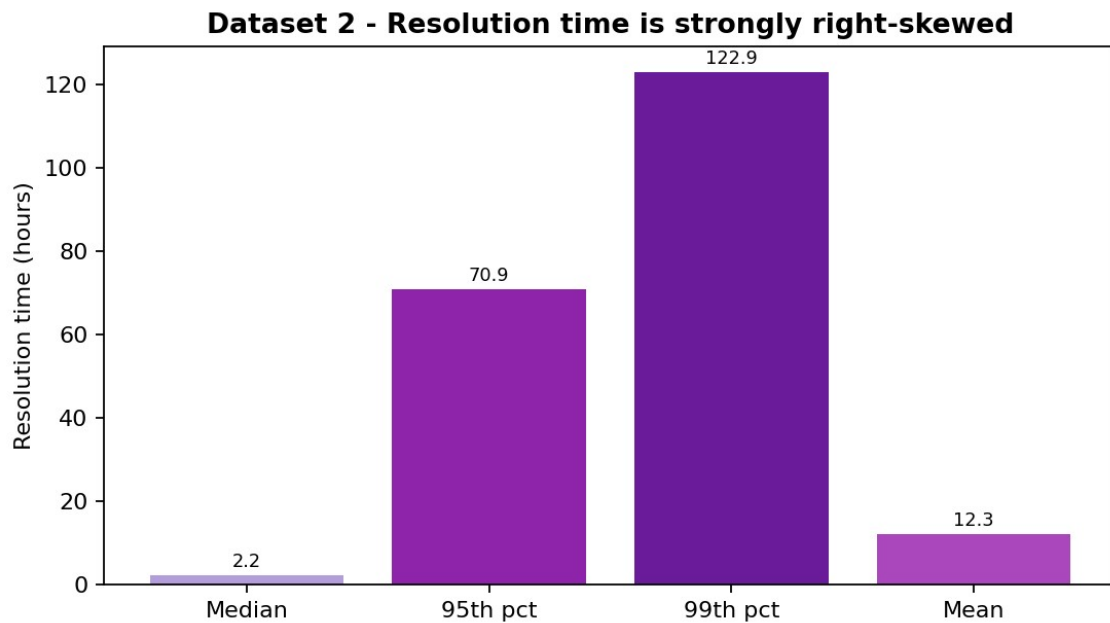


Figure 2.3. NYC 311 resolution-time distribution.

The resolution target is strongly right-skewed. Most cases close quickly, while a minority of requests take many days, creating large regression errors.

NYC 311 code pipeline and results

The notebook begins by locating the ZIP file and inspecting the internal CSV. It uses chunked profiling to avoid loading the full raw CSV into memory. It then creates a stratified sample by complaint category, audits distribution drift between the sample and the

original stream, resolves exact and Unique Key duplicates, constructs resolution_time_hours from Created Date and Closed Date, and engineers temporal features such as hour, day of week, month, weekend, after-hours, and cyclic time encodings. Geographic and channel features are also created from borough, ZIP, latitude/longitude, and Open Data Channel Type.

NYC 311 sampling integrity table

complaint_category	original_count	original_rate_within_selected	sample_count	sample_rate_within_selected	absolute_rate_difference
Noise - Street/Sidewalk	37968	0.0808	9699	0.0808	0.0000
General Construction/Plumbing	8059	0.0172	2059	0.0172	0.0000
Noise - Commercial	12713	0.0271	3247	0.0271	0.0000
Water System	15007	0.0319	3833	0.0319	0.0000
Illegal Parking	114730	0.2442	29307	0.2442	0.0000
PAINT/PLASTER	11002	0.0234	2810	0.0234	0.0000
UNSANITARY CONDITION	22501	0.0479	5748	0.0479	0.0000
Blocked Driveway	33152	0.0706	8468	0.0706	0.0000
Abandoned Vehicle	14276	0.0304	3647	0.0304	0.0000
Noise	11841	0.0252	3025	0.0252	0.0000
PLUMBING	13112	0.0279	3349	0.0279	0.0000
HEAT/HOT WATER	22073	0.0470	5638	0.0470	0.0000
DOOR/WINDOW	8422	0.0179	2151	0.0179	0.0000
Noise - Vehicle	11311	0.0241	2889	0.0241	0.0000
Noise - Residential	68814	0.1465	17578	0.1465	0.0000
Damaged Tree	10730	0.0228	2741	0.0228	0.0000
Traffic Signal Condition	8021	0.0171	2049	0.0171	0.0000
Street Condition	24761	0.0527	6325	0.0527	0.0000
Dirty Condition	11666	0.0248	2980	0.0248	0.0000
Encampment	9611	0.0205	2455	0.0205	0.0000

Missingness is operationally meaningful: Closed Date is missing for about 10.97% of profiled records, which usually means the request is open or unresolved rather than randomly missing. The notebook treats this as MAR/MNAR-like operational missingness rather than simple noise. Duplicate Unique Key rows were 0, indicating that the sampled data did not contain repeated official IDs after the duplicate-resolution stage.

NYC 311 modeling interpretation

The NYC notebook implements from-scratch non-kernel and kernel models. For this dataset, temporal/geographic variables make nonlinear kernels attractive because complaint dynamics can change by location, time of day, season, and channel. However, the kernel matrix is expensive: even moderate sample sizes require $O(n^2)$ memory and KRR/KPCA can require $O(n^3)$ decomposition or inversion. This is exactly why the notebook uses intelligent sampling and why the report emphasizes computational feasibility rather than blindly maximizing data volume.

Code workflow walkthrough - Dataset 2 - NYC 311

This table explains the notebook workflow section by section. It is included because the assignment grades the reasoning process and data engineering pipeline, not only final metrics. The notebook contains 51 documented sections.

Section	Notebook section title	What it does
1	Homework 3 — Dataset 2: NYC 311 Service Requests	Supports the dataset-specific pipeline and documents an intermediate result.
2	Section 0 — Project Setup	Supports the dataset-specific pipeline and documents an intermediate result.
3	Section 1 — Locate and Inspect the NYC 311 ZIP File	Supports the dataset-specific pipeline and documents an intermediate result.
4	Section 2 — Read a Small Preview and Build the Initial Schema	Inspects dtypes, missing rates, unique values, and raw structure before cleaning.
5	Section 3 — Dataset-Specific Target Definition	Defines and validates supervised learning targets while preventing target leakage.
6	Section 4 — Chunked First-Pass Profiling	Supports the dataset-specific pipeline and documents an intermediate result.
7	Section 5 — Intelligent Stratified Sampling	Supports the dataset-specific pipeline and documents an intermediate result.
8	Section 6 — Sampling Integrity Check	Supports the dataset-specific pipeline and documents an intermediate result.
9	Section 7 — Duplicate Report Resolution	Creates report-ready outputs, final audit checks, and deliverable indexes.
10	Section 8 — Target Engineering: Resolution Time	Defines and validates supervised learning targets while preventing target leakage.
11	Section 9 — Temporal Feature Engineering	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
12	Section 10 — Geographic and Channel Feature Engineering	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
13	Section 11 — Missing Value Analysis	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
14	Section 12 — Missingness Indicators and MCAR/MAR/MNAR Evidence	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
15	Section 13 — Evidence-Based Missingness Classification	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
16	Section 14 — Imputation Strategy Design	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.

ML Homework 3 - Expanded Four-Dataset Report

17	Section 15 — Outlier Analysis Overview	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
18	Section 16 — Z-score and IQR Outlier Detection	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
19	Section 17 — Simple Isolation Forest from Scratch	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
20	Section 18 — Compare Outlier Detection Methods	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
21	Section 19 — Feature Quality Analysis Overview	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
22	Section 20 — Near-Constant and High-Cardinality Features	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
23	Section 21 — Redundancy and Leakage Analysis	Supports the dataset-specific pipeline and documents an intermediate result.
24	Section 22 — Final Feature Decision Table	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
25	Section 23 — Modeling Dataset Construction	Implements/evaluates from-scratch non-kernel and kernel learning models.
26	Section 24 — Train/Test Split from Scratch	Builds train/test matrices with from-scratch preprocessing.
27	Section 25 — From-Scratch Tabular Preprocessor	Builds train/test matrices with from-scratch preprocessing.
28	Section 26 — Build Final Modeling Matrices	Implements/evaluates from-scratch non-kernel and kernel learning models.
29	Section 27 — Metrics from Scratch	Supports the dataset-specific pipeline and documents an intermediate result.
30	Section 28 — Modeling Subsamples for Expensive Algorithms	Implements/evaluates from-scratch non-kernel and kernel learning models.
31	Section 29 — Linear Regression from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
32	Section 30 — Multiclass Logistic Regression from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
33	Section 31 — KNN from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
34	Section 32 — Decision Tree from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
35	Section 33 — Kernel Functions and Kernel Ridge Regression	Runs kernel experiments and explains why each kernel works or fails on the dataset.
36	Section 34 — Kernel KNN from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
37	Section 35 — Simplified One-vs-Rest Kernel SVM from Scratch	Runs kernel experiments and explains why each kernel works or fails on the dataset.
38	Section 36 — KPCA + Downstream Classifier	Runs kernel experiments and explains why each kernel works or fails on the dataset.
39	Section 37 — Consolidated Results	Supports the dataset-specific pipeline and documents an intermediate result.
40	Section 38 — Kernel Investigation Discussion Table	Runs kernel experiments and explains why each kernel works or fails on the dataset.
41	Section 39 — Failure Analysis	Audits the worst predictions and distinguishes model limitation from data-quality artifacts.
42	Section 40 — Bias and Fairness Analysis	Analyzes group-level reliability, sampling bias, temporal drift, and geographic/customer bias.
43	Section 41 — Computational Analysis	Discusses runtime $O(n^2)/O(n^3)$ limits.
44	Section 42 — Theory vs Reality Discussion	Supports the dataset-specific pipeline and documents an intermediate result.
45	Section 43 — Final Report-Ready Summary for Dataset 2	Creates report-ready outputs, final audit checks, and deliverable indexes.
46	Section 44 — Save Final Cleaned Dataset Snapshot	Supports the dataset-specific pipeline and documents an intermediate result.
47	Section 45 — Final Assignment Compliance Checklist	Creates report-ready outputs, final audit checks, and deliverable indexes.
48	Section 46 — Final Leakage and Reproducibility Audit	Supports the dataset-specific pipeline and documents an intermediate result.
49	Section 47 — Final Model Selection and Interpretation Tables	Implements/evaluates from-scratch non-kernel and kernel learning models.
50	Section 48 — Mini-Conference Paper Section for Dataset 2	Supports the dataset-specific pipeline and documents an intermediate result.
51	Section 49 — Final Deliverable Package Index	Creates report-ready outputs, final audit checks, and deliverable indexes.

5. Dataset 3 - IBM HR Employee Attrition

The HR Attrition dataset is a small but high-risk structured classification problem. Unlike Airbnb and NYC, it does not have many missing values or huge row counts. Its central challenge is label imbalance and interpretation: the positive class Attrition is much smaller than the negative class, and the label does not explain why the employee left. The assignment specifically required root-driver investigation, rigorous feature selection, and sampling strategies such as SMOTE or undersampling.

HR class imbalance table

class_label	count	rate	class_name
0	1233	0.8388	No Attrition
1	237	0.1612	Attrition

The attrition rate is only 16.12%, meaning that a naive model can obtain high accuracy by predicting No Attrition. Therefore, the notebook emphasizes precision, recall, F1, and balanced accuracy rather than raw accuracy alone.

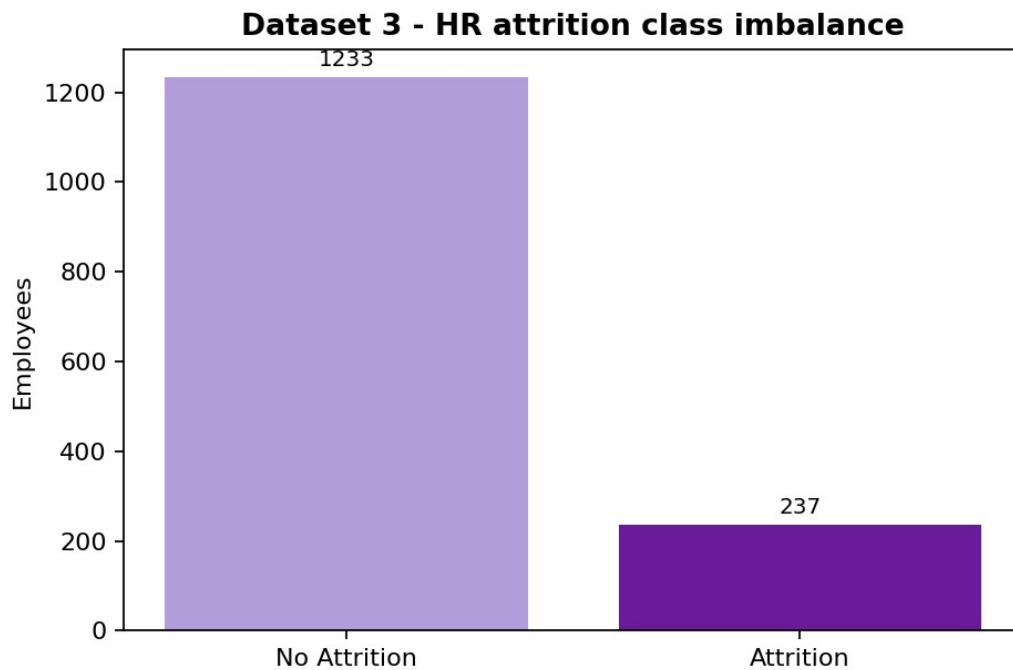


Figure 3.1. HR class imbalance.

The minority attrition class is small enough that accuracy alone is misleading. This explains why SMOTE and undersampling were used.

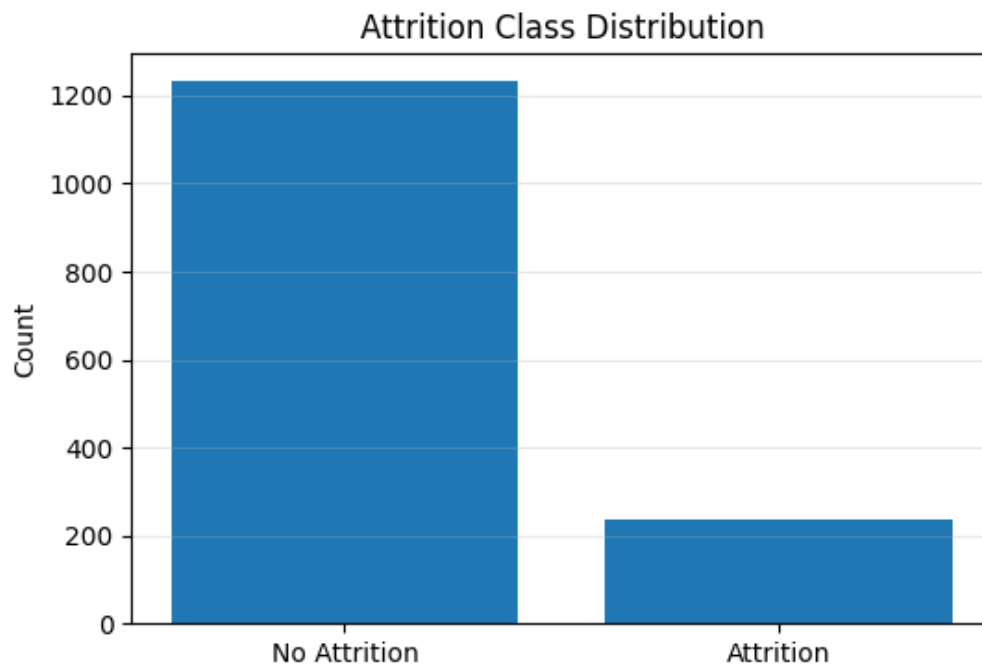


Figure 3.2. Notebook class imbalance plot.

This plot comes directly from the HR notebook and visually confirms the No/Yes imbalance in the Attrition target.

HR root drivers and feature selection

Top numerical HR attrition drivers

ML Homework 3 - Expanded Four-Dataset Report

feature	mean_no_attrition	mean_attrition	median_no_attrition	median_attrition	standardized_mean_difference_yes_minus_no	abs_effect_size	mann_whitney_p_value
overtime_binary	0.2344	0.5359	0.0000	1.0000	0.6690	0.6690	0.0000
overtime_binary	0.2344	0.5359	0.0000	1.0000	0.6690	0.6690	0.0000
early_career_flag	0.0827	0.2658	0.0000	0.0000	0.5798	0.5798	0.0000
early_career_flag	0.0827	0.2658	0.0000	0.0000	0.5798	0.5798	0.0000
simple_iforest_score	0.4919	0.5088	0.4897	0.5086	0.5054	0.5054	0.0000
TotalWorkingYears	11.8629	8.2447	10.0000	7.0000	-0.4650	0.4650	0.0000
JobLevel	2.1460	1.6371	2.0000	1.0000	-0.4597	0.4597	0.0000
YearsInCurrentRole	4.4842	2.9030	3.0000	2.0000	-0.4364	0.4364	0.0000
MonthlyIncome	6832.7397	4787.0928	5204.0000	3202.0000	-0.4345	0.4345	0.0000
Age	37.5612	33.6076	36.0000	32.0000	-0.4328	0.4328	0.0000
role_tenure_ratio	0.4996	0.3822	0.5294	0.4286	-0.4285	0.4285	0.0000
role_tenure_ratio	0.4996	0.3822	0.5294	0.4286	-0.4285	0.4285	0.0000
YearsWithCurrManager	4.3674	2.8523	3.0000	2.0000	-0.4246	0.4246	0.0000
manager_tenure_ratio	0.4841	0.3688	0.5000	0.4667	-0.4166	0.4166	0.0000
manager_tenure_ratio	0.4841	0.3688	0.5000	0.4667	-0.4166	0.4166	0.0000

Top categorical HR attrition drivers

feature	unique_categories	cramers_v	chi_square_p_value	highest_attrition_category	highest_attrition_rate	lowest_attrition_category	lowest_attrition_rate
OverTime	2	0.2441	0.0000	Yes	0.3053	No	0.1044
JobRole	9	0.2421	0.0000	Sales Representative	0.3976	Research Director	0.0250
MaritalStatus	3	0.1772	0.0000	Single	0.2553	Divorced	0.1009
BusinessTravel	3	0.1283	0.0000	Travel_Frequently	0.2491	Non-Travel	0.0800
EducationField	6	0.1044	0.0068	Human Resources	0.2593	Other	0.1341
Department	3	0.0857	0.0045	Sales	0.2063	Research & Development	0.1384
Gender	2	0.0276	0.2906	Male	0.1701	Female	0.1480

Most influential Logistic Regression coefficients

Unnamed: 0	encoded_feature	coefficient	abs_coefficient
0	OverTime__No	-0.9458	0.9458
1	BusinessTravel__Non-Travel	-0.7736	0.7736
2	EnvironmentSatisfaction	-0.6301	0.6301
3	MaritalStatus__Divorced	-0.6191	0.6191
4	JobSatisfaction	-0.6099	0.6099
5	NumCompaniesWorked	0.6044	0.6044
6	YearsSinceLastPromotion	0.5695	0.5695
7	Gender__Female	-0.4970	0.4970
8	overtime_binary	0.4859	0.4859
9	OverTime__Yes	0.4859	0.4859
10	Department__Research & Development	-0.4645	0.4645
11	TotalWorkingYears	-0.4640	0.4640
12	JobInvolvement	-0.4591	0.4591
13	JobRole__Laboratory Technician	0.4423	0.4423
14	BusinessTravel__Travel_Frequently	0.4094	0.4094

The root-driver analysis shows that attrition is associated with features such as overtime, travel, satisfaction, distance from home, tenure, number of prior companies, and promotion stagnation. These are treated as associations, not causal claims, because the label is historical and does not distinguish voluntary resignation from involuntary termination.

HR modeling results

HR non-kernel model results

model	training_variant	runtime_seconds	accuracy	precision	recall	specificity	f1	balanced_accuracy	tp	tn	fp	fn
Logistic Regression	smote	0.0628	0.7778	0.4035	0.7667	0.7799	0.5287	0.7733	46	241	68	14
Logistic Regression	undersampled	0.0231	0.7561	0.3828	0.8167	0.7443	0.5213	0.7805	49	230	79	11
Logistic Regression	oversampled	0.0641	0.7696	0.3932	0.7667	0.7702	0.5198	0.7684	46	238	71	14
Logistic Regression	original	0.0442	0.8808	0.7667	0.3833	0.9773	0.5111	0.6803	23	302	7	37
Decision Tree	oversampled	0.1786	0.7236	0.3056	0.5500	0.7573	0.3929	0.6536	33	234	75	27
Decision Tree	smote	0.5591	0.7805	0.3562	0.4333	0.8479	0.3910	0.6406	26	262	47	34
KNN	undersampled	0.0144	0.7480	0.3146	0.4667	0.8026	0.3758	0.6346	28	248	61	32
KNN	smote	0.0865	0.5799	0.2432	0.7500	0.5469	0.3673	0.6485	45	169	140	15
KNN	oversampled	0.0742	0.7398	0.3000	0.4500	0.7961	0.3600	0.6231	27	246	63	33
Decision Tree	undersampled	0.0950	0.6992	0.2661	0.4833	0.7411	0.3432	0.6122	29	229	80	31
Decision Tree	original	0.1588	0.8184	0.4054	0.2500	0.9288	0.3093	0.5894	15	287	22	45
KNN	original	0.0425	0.8401	0.6000	0.0500	0.9935	0.0923	0.5218	3	307	2	57

HR kernel model results

ML Homework 3 - Expanded Four-Dataset Report

Unnamed: 0	model	kernel	runtime_seconds	accuracy	precision	recall	specificity	f1	balanced_accuracy	tp	tn	fp	fn
0	Kernel Ridge Classifier	linear	0.1786	0.7724	0.3966	0.7667	0.7735	0.5227	0.7701	46	239	70	14
1	Kernel Ridge Classifier	rbf	0.2759	0.8103	0.4419	0.6333	0.8447	0.5205	0.7390	38	261	48	22
2	Kernel Ridge Classifier	poly_degree_2	0.2361	0.7967	0.4194	0.6500	0.8252	0.5098	0.7376	39	255	54	21
3	Kernel SVM	linear	4.5487	0.7398	0.3594	0.7667	0.7346	0.4894	0.7506	46	227	82	14
4	Kernel SVM	poly_degree_2	3.5428	0.7561	0.3707	0.7167	0.7638	0.4886	0.7402	43	236	73	17
5	Kernel SVM	rbf	2.7752	0.7642	0.3714	0.6500	0.7864	0.4727	0.7182	39	243	66	21
6	KPCA + Logistic	linear	1.5056	0.7344	0.3443	0.7000	0.7411	0.4615	0.7206	42	229	80	18
7	KPCA + Logistic	poly_degree_2	1.5567	0.6911	0.2955	0.6500	0.6990	0.4062	0.6745	39	216	93	21
8	KPCA + Logistic	rbf	1.9610	0.6721	0.2837	0.6667	0.6731	0.3980	0.6699	40	208	101	20
9	Kernel KNN	linear	0.0436	0.5799	0.2432	0.7500	0.5469	0.3673	0.6485	45	169	140	15
10	Kernel KNN	rbf	0.1290	0.5799	0.2432	0.7500	0.5469	0.3673	0.6485	45	169	140	15
11	Kernel KNN	poly_degree_2	0.0792	0.5583	0.2359	0.7667	0.5178	0.3608	0.6422	46	160	149	14

HR combined model comparison

Unnamed: 0	model	training_variant	runtime_seconds	accuracy	precision	recall	specificity	f1	balanced_accuracy	tp	tn	fp	fn	kernel	model_family
0	Logistic Regression	smote	0.2463	0.7778	0.4035	0.7667	0.7799	0.5287	0.7733	46	241	68	14	none	non_kernel
1	Kernel Ridge Classifier	smote_or_subset	0.1786	0.7724	0.3966	0.7667	0.7735	0.5227	0.7701	46	239	70	14	linear	kernel
2	Logistic Regression	undersampled	0.0677	0.7561	0.3828	0.8167	0.7443	0.5213	0.7805	49	230	79	11	none	non_kernel
3	Kernel Ridge Classifier	smote_or_subset	0.2759	0.8103	0.4419	0.6333	0.8447	0.5205	0.7390	38	261	48	22	rbf	kernel
4	Logistic Regression	oversampled	0.1658	0.7696	0.3932	0.7667	0.7702	0.5198	0.7684	46	238	71	14	none	non_kernel
5	Logistic Regression	original	0.1297	0.8808	0.7667	0.3833	0.9773	0.5111	0.6803	23	302	7	37	none	non_kernel
6	Kernel Ridge Classifier	smote_or_subset	0.2361	0.7967	0.4194	0.6500	0.8252	0.5098	0.7376	39	255	54	21	poly_degree_2	kernel
7	Kernel SVM	smote_or_subset	4.5487	0.7398	0.3594	0.7667	0.7346	0.4894	0.7506	46	227	82	14	linear	kernel
8	Kernel SVM	smote_or_subset	3.5428	0.7561	0.3707	0.7167	0.7638	0.4886	0.7402	43	236	73	17	poly_degree_2	kernel
9	Kernel SVM	smote_or_subset	2.7752	0.7642	0.3714	0.6500	0.7864	0.4727	0.7182	39	243	66	21	rbf	kernel
10	KPCA + Logistic	smote_or_subset	1.5056	0.7344	0.3443	0.7000	0.7411	0.4615	0.7206	42	229	80	18	linear	kernel
11	KPCA + Logistic	smote_or_subset	1.5567	0.6911	0.2955	0.6500	0.6990	0.4062	0.6745	39	216	93	21	poly_degree_2	kernel

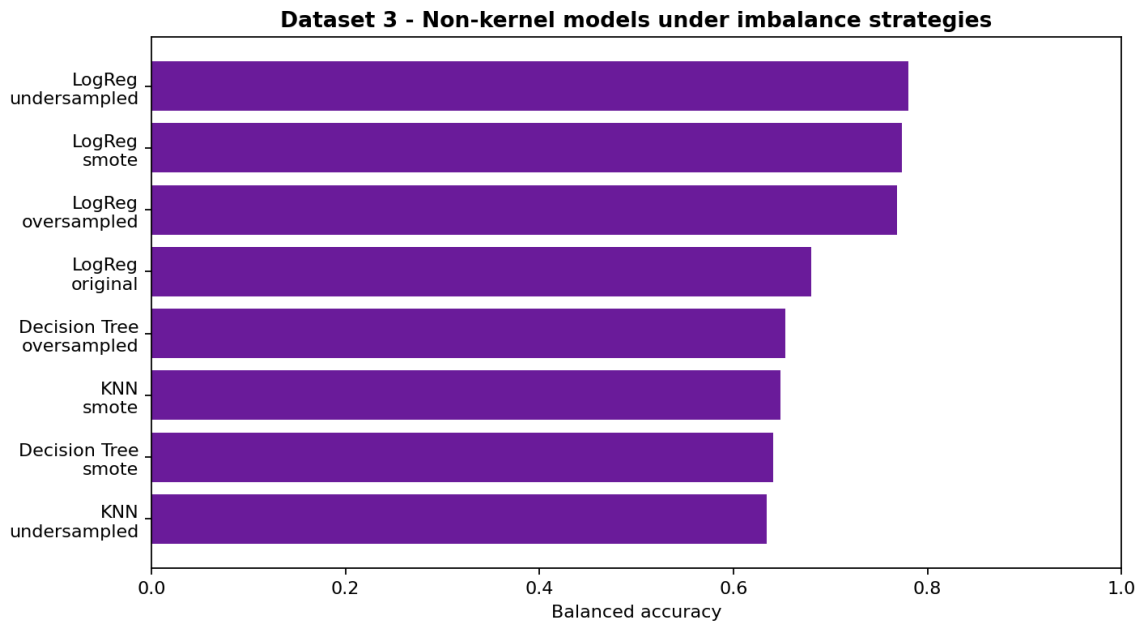
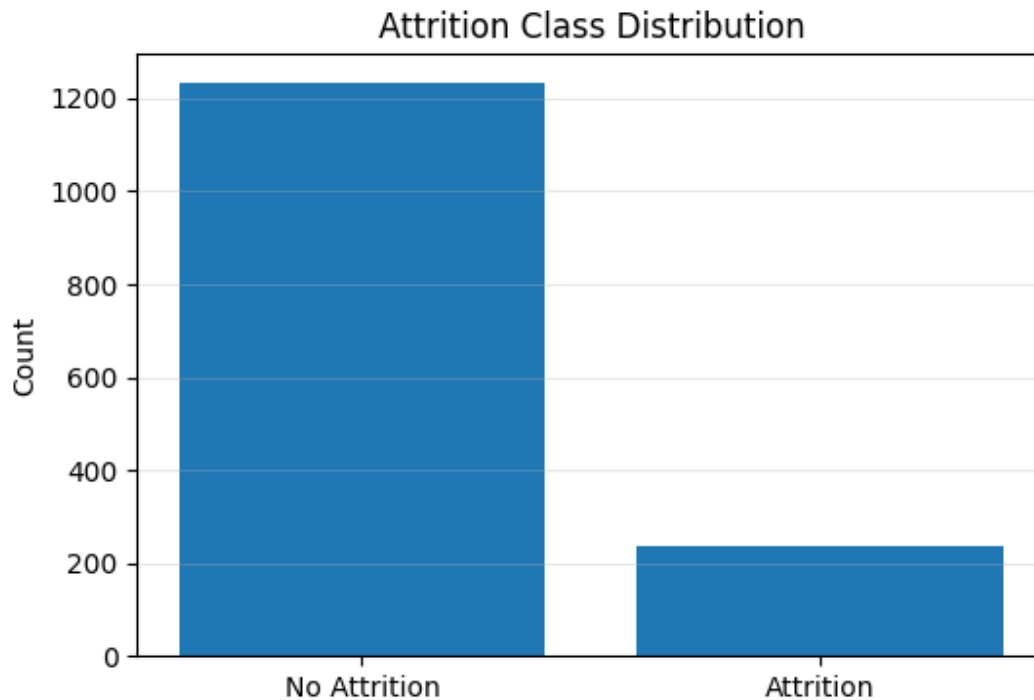


Figure 3.3. HR balanced accuracy by model.

Balanced accuracy is the correct summary metric for HR because the positive attrition class is rare. SMOTE and undersampling improve minority-class recall relative to the original training distribution.

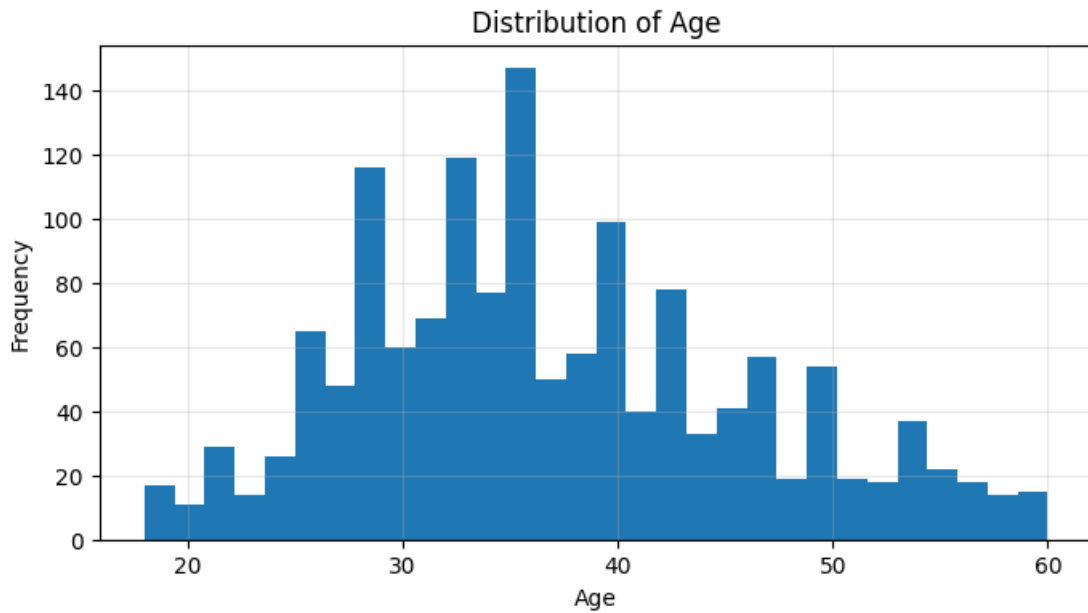
The best practical model chosen in the notebook is Logistic Regression with SMOTE. Its accuracy is lower than the original-distribution Logistic Regression, but its recall is much higher. In HR attrition, missing a true attrition case is operationally costly, so the recall improvement is more important than maximizing accuracy.

HR notebook figures and detailed interpretation



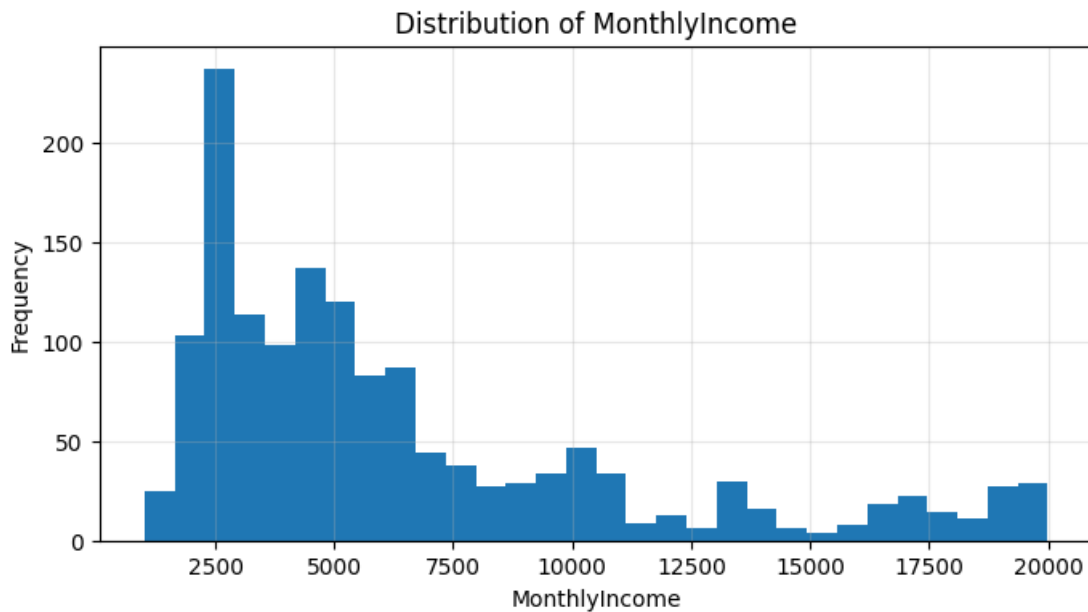
HR Figure 1. Class imbalance

The chart proves the class-imbalance problem directly: No Attrition dominates the target, so recall and balanced accuracy are required.



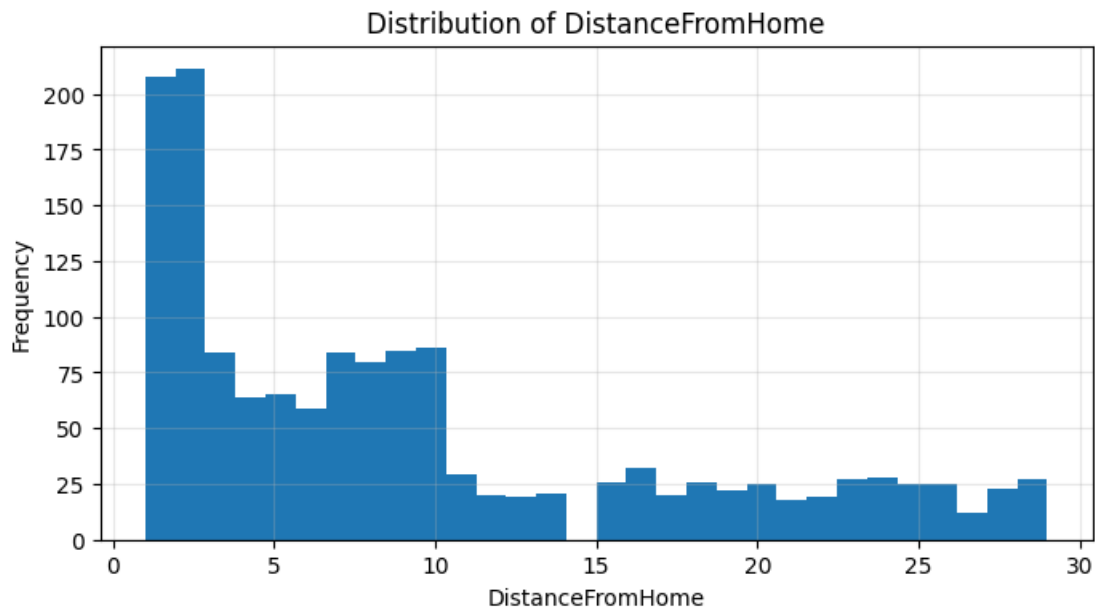
HR Figure 2. Age

This HR figure shows the distribution of Age. It supports either imbalance handling or feature-quality analysis by showing skew, concentration, or limited support in the HR variable.



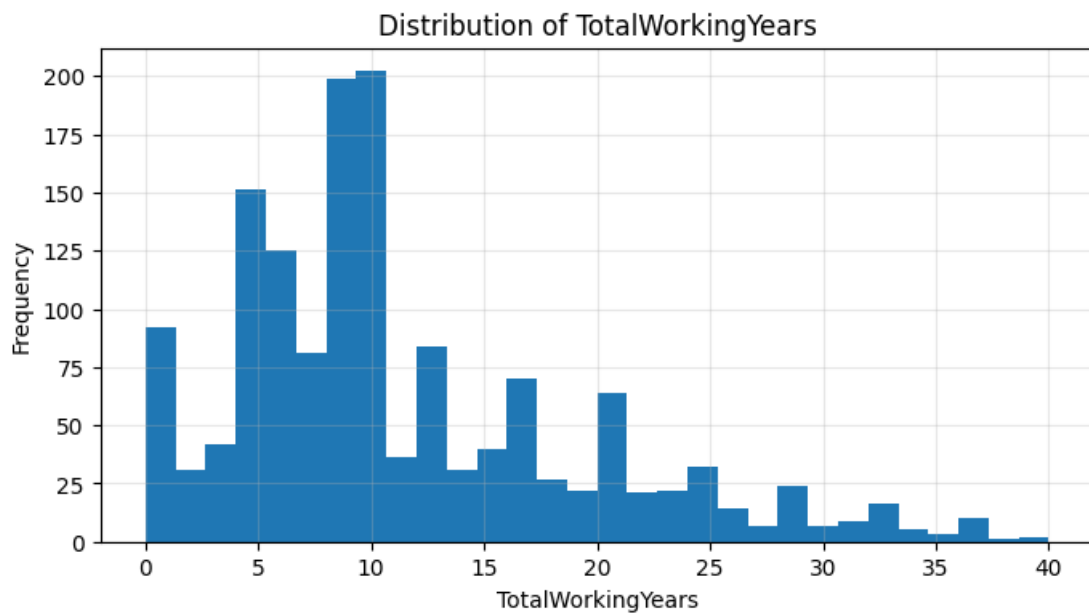
HR Figure 3. MonthlyIncome

This HR figure shows the distribution of MonthlyIncome. It supports either imbalance handling or feature-quality analysis by showing skew, concentration, or limited support in the HR variable.



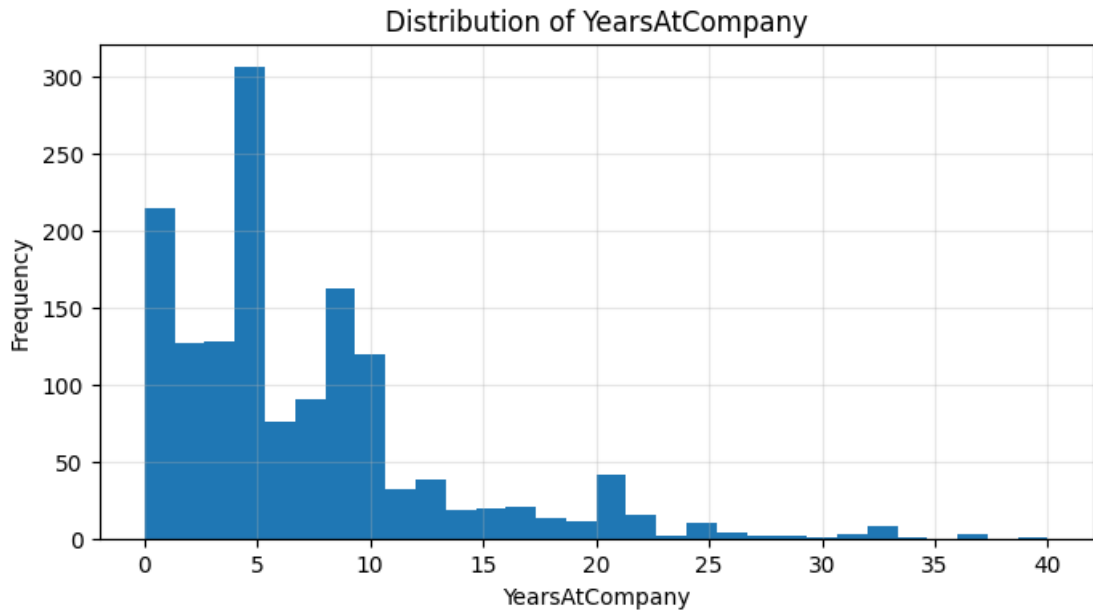
HR Figure 4. DistanceFromHome

This HR figure shows the distribution of DistanceFromHome. It supports either imbalance handling or feature-quality analysis by showing skew, concentration, or limited support in the HR variable.



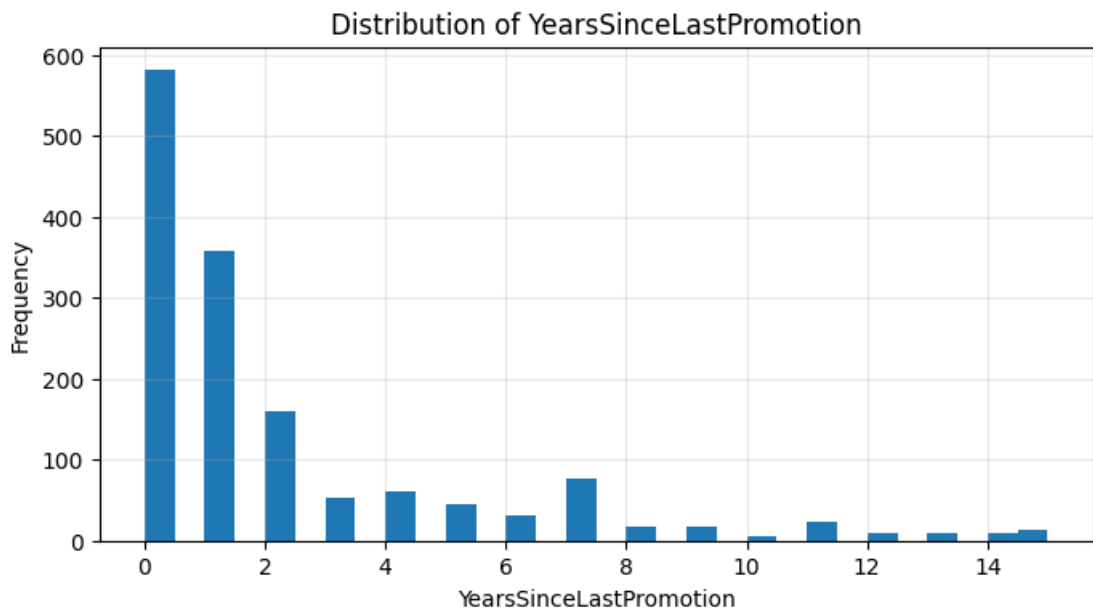
HR Figure 5. TotalWorkingYears

This HR figure shows the distribution of TotalWorkingYears. It supports either imbalance handling or feature-quality analysis by showing skew, concentration, or limited support in the HR variable.



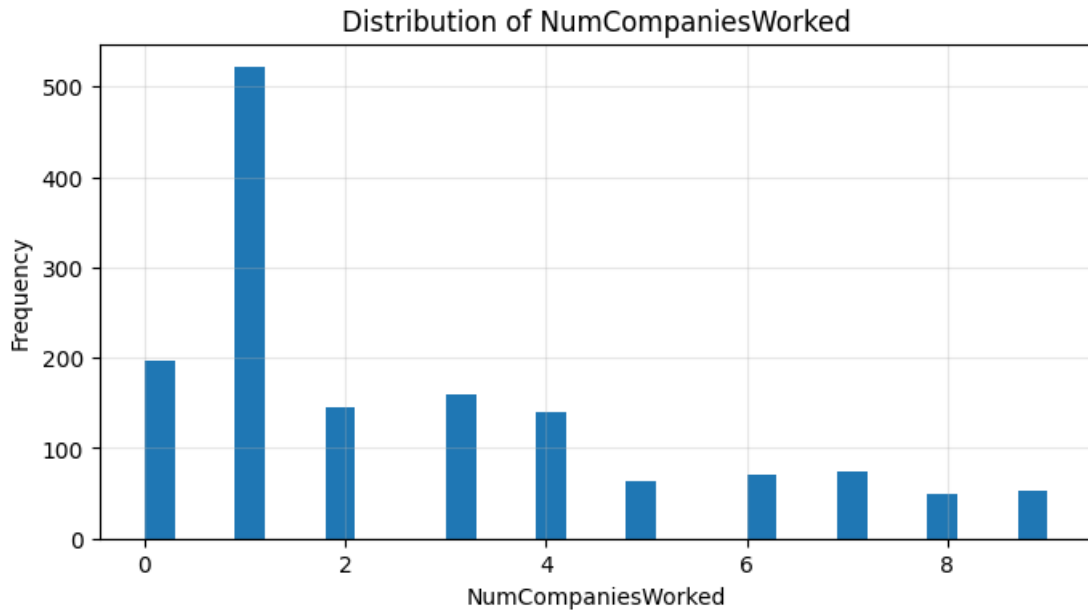
HR Figure 6. YearsAtCompany

This HR figure shows the distribution of YearsAtCompany. It supports either imbalance handling or feature-quality analysis by showing skew, concentration, or limited support in the HR variable.



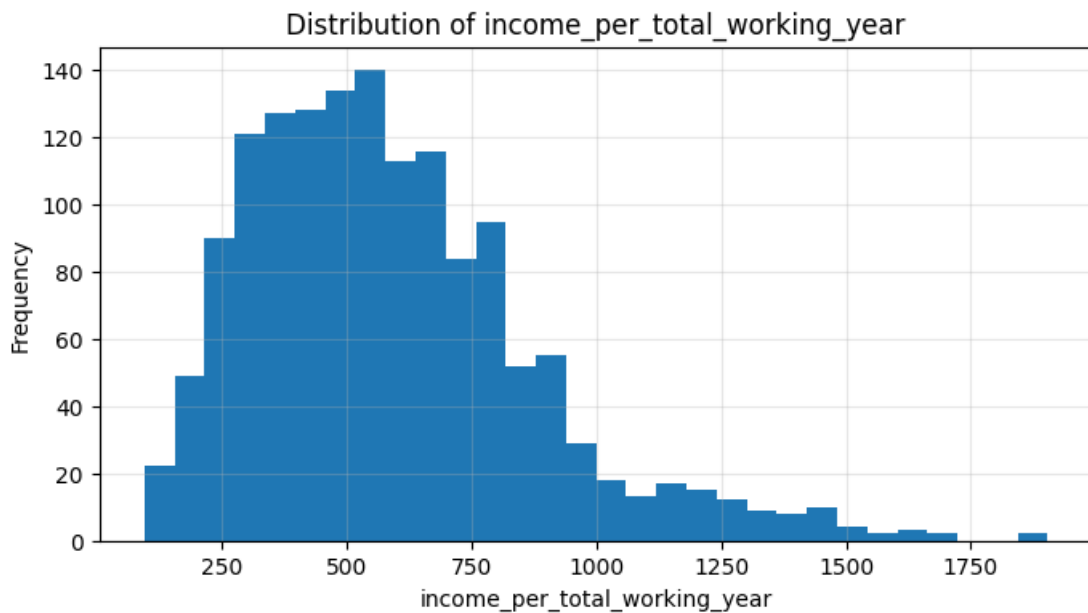
HR Figure 7. YearsSinceLastPromotion

This HR figure shows the distribution of YearsSinceLastPromotion. It supports either imbalance handling or feature-quality analysis by showing skew, concentration, or limited support in the HR variable.



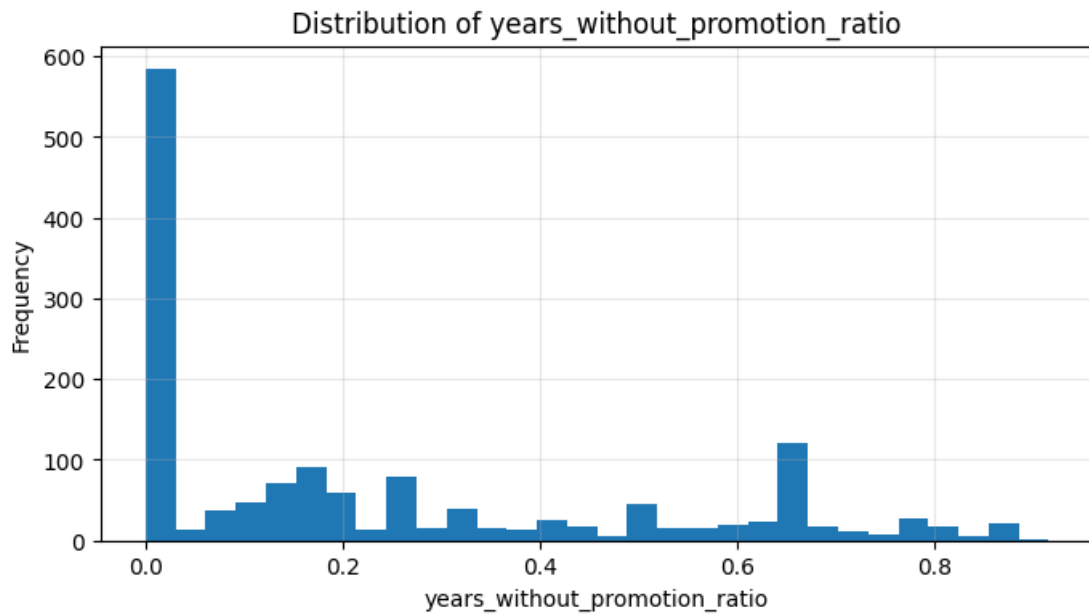
HR Figure 8. NumCompaniesWorked

This HR figure shows the distribution of NumCompaniesWorked. It supports either imbalance handling or feature-quality analysis by showing skew, concentration, or limited support in the HR variable.



HR Figure 9. income_per_total_working_year

This HR figure shows the distribution of income_per_total_working_year. It supports either imbalance handling or feature-quality analysis by showing skew, concentration, or limited support in the HR variable.



HR Figure 10. years_without_promotion_ratio

This HR figure shows the distribution of years_without_promotion_ratio. It supports either imbalance handling or feature-quality analysis by showing skew, concentration, or limited support in the HR variable.

HR bias and computation

HR group-level fairness and reliability audit

Unnamed: 0	group_column	group_value	n	actual_attrition_rate	predicted_attrition_rate	accuracy	precision	recall	specificity	f1	balance_d_accuracy	tp	tn	fp	fn
0	AgeGroup	26-35	159	0.2075	0.3459	0.7862	0.4909	0.8182	0.7778	0.6136	0.7980	27	98	28	6
1	AgeGroup	36-45	112	0.0982	0.2500	0.8125	0.3214	0.8182	0.8119	0.4615	0.8150	9	82	19	2
2	AgeGroup	46-55	54	0.0741	0.1667	0.7963	0.1111	0.2500	0.8400	0.1538	0.5450	1	42	8	3
3	AgeGroup	<=25	32	0.3125	0.6250	0.6250	0.4500	0.9000	0.5000	0.6000	0.7000	9	11	11	1
4	AgeGroup	56+	12	0.1667	0.1667	0.6667	0.0000	0.0000	0.8000	0.0000	0.4000	0	8	2	2
5	BusinessTravel	Travel_Rarely	261	0.1456	0.3065	0.7701	0.3625	0.7632	0.7713	0.4915	0.7672	29	172	51	9
6	BusinessTravel	Travel_Frequently	73	0.2603	0.3836	0.7671	0.5357	0.7895	0.7593	0.6383	0.7744	15	41	13	4
7	BusinessTravel	Non-Travel	35	0.0857	0.1714	0.8571	0.3333	0.6667	0.8750	0.4444	0.7708	2	28	4	1
8	Department	Research & Development	246	0.1504	0.3089	0.7764	0.3816	0.7838	0.7751	0.5133	0.7795	29	162	47	8
9	Department	Sales	107	0.1963	0.3364	0.7664	0.4444	0.7619	0.7674	0.5614	0.7647	16	66	20	5
10	Department	Human Resources	16	0.1250	0.1250	0.8750	0.5000	0.5000	0.9286	0.5000	0.7143	1	13	1	1
11	Gender	Male	229	0.1485	0.3231	0.7555	0.3514	0.7647	0.7538	0.4815	0.7593	26	147	48	8
12	Gender	Female	140	0.1857	0.2857	0.8143	0.5000	0.7692	0.8246	0.6061	0.7969	20	94	20	6
13	JobRole	Research Scientist	80	0.1750	0.3625	0.7625	0.4138	0.8571	0.7424	0.5581	0.7998	12	49	17	2
14	JobRole	Sales Executive	78	0.1538	0.2821	0.7692	0.3636	0.6667	0.7879	0.4706	0.7273	8	52	14	4
15	JobRole	Laboratory Technician	67	0.2537	0.5075	0.6866	0.4412	0.8824	0.6200	0.5882	0.7512	15	31	19	2
16	JobRole	Manufacturing Director	35	0.1429	0.1714	0.7429	0.1667	0.2000	0.8333	0.1818	0.5167	1	25	5	4
17	JobRole	Healthcare	30	0.0333	0.1667	0.8667	0.2000	1.0000	0.8621	0.3333	0.9310	1	25	4	0

		Representative													
--	--	----------------	--	--	--	--	--	--	--	--	--	--	--	--	--

HR kernel computational analysis

Unnamed: 0	dataset_variant	n_samples	n_features	kernel_matrix_memory_MB_float64	kernel_matrix_entries	approx_kernel_construction_complexity	approx_kernel_solve_complexity
0	original_train	1101	67	9.2484	1212201	$O(n^2 d)$	$O(n^3)$
1	undersampled_train	354	67	0.9561	125316	$O(n^2 d)$	$O(n^3)$
2	oversampled_train	1848	67	26.0552	3415104	$O(n^2 d)$	$O(n^3)$
3	smote_train	1848	67	26.0552	3415104	$O(n^2 d)$	$O(n^3)$
4	svm_subset	700	67	3.7384	490000	$O(n^2 d)$	$O(n^3)$

The HR fairness audit checks reliability by gender, age group, department, job role, marital status, business travel, and overtime. The computational table shows that HR is small enough for kernel methods, but the small sample size also means overfitting is a stronger risk.

Code workflow walkthrough - Dataset 3 - IBM HR Attrition

This table explains the notebook workflow section by section. It is included because the assignment grades the reasoning process and data engineering pipeline, not only final metrics. The notebook contains 50 documented sections.

Section	Notebook section title	What it does
1	Project 3 — Dataset 3: IBM HR Employee Attrition	Supports the dataset-specific pipeline and documents an intermediate result.
2	0. Project Setup	Supports the dataset-specific pipeline and documents an intermediate result.
3	1. Load the IBM HR Employee Attrition Dataset	Loads the raw dataset and verifies that files, columns, and row counts are correct.
4	2. Basic Schema and Data Type Inspection	Inspects dtypes, missing rates, unique values, and raw structure before cleaning.
5	3. Target Construction and Label Quality Analysis	Defines and validates supervised learning targets while preventing target leakage.
6	4. Class Imbalance Investigation	Supports the dataset-specific pipeline and documents an intermediate result.
7	5. Missing Data Analysis	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
8	6. Missingness Mechanism Classification	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
9	7. Duplicate and Identifier Analysis	Supports the dataset-specific pipeline and documents an intermediate result.
10	8. Constant and Near-Constant Feature Analysis	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
11	9. HR Domain Feature Engineering	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
12	10. Outlier Analysis Overview	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
13	11. Visualize Important Numeric Distributions	Supports the dataset-specific pipeline and documents an intermediate result.
14	12. Z-score Outlier Detection	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
15	13. IQR Outlier Detection	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
16	14. Simple Isolation Forest from Scratch	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
17	15. Run Simple Isolation Forest on HR Numeric Features	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
18	16. Compare Z-score, IQR, and Isolation Forest	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
19	17. Root Driver Analysis: Numerical Features	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
20	18. Root Driver Analysis: Categorical Features	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
21	19. Feature Quality Analysis	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
22	20. Final Feature Selection Table	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
23	21. Train/Test Split from Scratch	Builds train/test matrices with from-scratch preprocessing.
24	22. From-Scratch Tabular Preprocessor	Builds train/test matrices with from-scratch preprocessing.
25	23. Imbalance Handling from Scratch	Supports the dataset-specific pipeline and documents an intermediate result.
26	24. Evaluation Metrics from Scratch	Supports the dataset-specific pipeline and documents an intermediate result.
27	25. Logistic Regression from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
28	26. KNN Classifier from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
29	27. Decision Tree Classifier from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
30	28. Train Non-Kernel Models Across Sampling Strategies	Implements/evaluates from-scratch non-kernel and kernel learning models.
31	29. Kernel Functions from Scratch	Runs kernel experiments and explains why each kernel works or fails on the dataset.
32	30. Kernel Ridge Classification from Scratch	Runs kernel experiments and explains why each kernel works or fails on the dataset.
33	31. Kernel KNN from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
34	32. Kernel SVM from Scratch	Runs kernel experiments and explains why each kernel works or

ML Homework 3 - Expanded Four-Dataset Report

35	33. Kernel PCA from Scratch + Logistic Regression	fails on the dataset. Implements/evaluates from-scratch non-kernel and kernel learning models.
36	34. Train Kernel Models	Implements/evaluates from-scratch non-kernel and kernel learning models.
37	35. Combined Model Results	Implements/evaluates from-scratch non-kernel and kernel learning models.
38	36. Kernel Investigation Discussion Table	Runs kernel experiments and explains why each kernel works or fails on the dataset.
39	37. Model-Based Root Driver Analysis Using Logistic Coefficients	Implements/evaluates from-scratch non-kernel and kernel learning models.
40	38. Failure Analysis	Audits the worst predictions and distinguishes model limitation from data-quality artifacts.
41	39. Bias and Fairness Analysis	Analyzes group-level reliability, sampling bias, temporal drift, and geographic/customer bias.
42	40. Computational Analysis	Discusses runtime, memory, and kernel matrix $O(n^2)/O(n^3)$ limits.
43	41. Theory vs Reality Discussion	Supports the dataset-specific pipeline and documents an intermediate result.
44	42. Save Final Modeling Arrays and Outputs	Implements/evaluates from-scratch non-kernel and kernel learning models.
45	43. Report-Ready Final Summary	Creates report-ready outputs, final audit checks, and deliverable indexes.
46	44. Final Assignment Compliance Checklist	Creates report-ready outputs, final audit checks, and deliverable indexes.
47	45. Final Leakage and Reproducibility Audit	Supports the dataset-specific pipeline and documents an intermediate result.
48	46. Final Model Selection, Interpretation, and Report Tables	Implements/evaluates from-scratch non-kernel and kernel learning models.
49	47. Mini-Conference Paper Section for Dataset 3	Supports the dataset-specific pipeline and documents an intermediate result.
50	48. Final Deliverable Package Index	Creates report-ready outputs, final audit checks, and deliverable indexes.

6. Dataset 4 - Online Retail Transactions

The Online Retail dataset is fundamentally different from the other three because the raw rows are not independent supervised-learning examples. They are transaction-line records. The assignment required converting raw transactional logs into aggregated customer-level behavioral features before modeling customer segment and future spending. The notebook therefore creates a temporal split: historical transactions become features and future transactions become labels. This is the central leakage-control design of Dataset 4.

Online Retail transaction cleaning and customer aggregation

Online Retail raw schema summary

Unnamed: 0	column	dtype	non_null_count	missing_count	missing_rate	unique_values	unique_rate
0	CustomerID	float64	406829	135080	0.2493	4373	0.0081
1	Description	object	540455	1454	0.0027	4224	0.0078
2	StockCode	object	541909	0	0.0000	4070	0.0075
3	InvoiceNo	object	541909	0	0.0000	25900	0.0478
4	Quantity	int64	541909	0	0.0000	722	0.0013
5	InvoiceDate	datetime64[ns]	541909	0	0.0000	23260	0.0429
6	UnitPrice	float64	541909	0	0.0000	1630	0.0030
7	Country	object	541909	0	0.0000	38	0.0001

Online Retail missing-value summary

Unnamed: 0	column	dtype	missing_count	missing_rate	unique_values	missing_group
0	CustomerID	float64	135080	0.2493	4373	Moderate missingness
1	Description	object	1454	0.0027	4224	Low missingness
2	InvoiceNo	object	0	0.0000	25900	No missing
3	Quantity	int64	0	0.0000	722	No missing
4	StockCode	object	0	0.0000	4070	No missing
5	InvoiceDate	datetime64[ns]	0	0.0000	23260	No missing
6	UnitPrice	float64	0	0.0000	1630	No missing
7	Country	object	0	0.0000	38	No missing
8	InvoiceNo_str	object	0	0.0000	25900	No missing
9	StockCode_str	object	0	0.0000	4070	No missing
10	Description_str	object	0	0.0000	4224	No missing
11	is_cancel_invoice	int64	0	0.0000	2	No missing

Online Retail label-quality risks

Unnamed: 0	label_quality_issue	impact_on_modeling	mitigation
0	Future inactive may be right-censored	A customer may purchase after the observed fut...	Report limitation and analyze errors for inact...
1	Future period may contain seasonality	Models may learn seasonal patterns specific to...	Include temporal discussion and month features.

ML Homework 3 - Expanded Four-Dataset Report

2	Country distribution is imbalanced	Performance may be dominated by United Kingdom...	Evaluate metrics by country group.
3	Segments are quantile-derived, not externally ...	Segment boundaries are operational definitions...	Clearly document the label construction rules.
4	Historical features are not equally long for a...	Newer customers have less observed behavioral ...	Include customer_age_days and purchase rate fe...



Figure 4.1. Online Retail missing-value analysis.

The missingness plot shows that CustomerID and Description are major missingness sources. Missing CustomerID is especially important because customer-level modeling requires known customers.

The notebook removes or separates canceled invoices, negative quantities, non-positive prices, missing customer identifiers, and exact duplicates. It then aggregates invoice and customer features: history_total_spending, invoice count, unique products, recency, customer age, purchase rate, average invoice value, RFM scores, dominant country, and historical RFM segment.

Online Retail outlier analysis

Online Retail future columns excluded from outlier analysis

Unnamed: 0	column	reason_excluded_from_outlier_analysis
0	future_spending	future target or target-derived column
1	future_invoice_count	future target or target-derived column
2	future_line_count	future target or target-derived column
3	future_quantity	future target or target-derived column
4	future_unique_products	future target or target-derived column
5	log1p_future_spending	future target or target-derived column
6	future_active	future target or target-derived column
7	future_customer_segment	future target or target-derived column
8	future_segment_id	future target or target-derived column

Online Retail customer-level numeric outlier profile

Unnamed: 0	column	non_missing_count	mean	median	std	min	p01	p25	p75	p99	max	skewness
0	history_median_line_value	3370	52.0023	15.8000	1334.3338	0.7800	2.1000	9.7500	18.7150	321.9840	77,183.60	57.3828
1	history_avg_line_value	3370	61.6490	17.8259	1353.8033	1.4500	3.2156	12.7500	25.5508	361.7174	77,183.60	55.2000
2	avg_invoice_quantity	3370	240.8080	149.0000	1309.0022	1.0000	8.6900	83.2750	257.9125	1385.6609	74,215.00	53.6891
3	quantity_per_invoice	3370	240.8080	149.0000	1309.0022	1.0000	8.6900	83.2750	257.9125	1385.6609	74,215.00	53.6891
4	median_invoice_value	3370	379.4854	279.8400	1396.5225	2.9000	35.6760	163.2125	403.9712	1910.0561	77,183.60	49.8540
5	monetary_per_invoice	3370	397.3177	283.0770	1422.6305	2.9000	38.8038	168.3025	412.4110	2032.9781	77,183.60	47.3696
6	avg_invoice_value	3370	397.3177	283.0770	1422.6305	2.9000	38.8038	168.3025	412.4110	2032.9781	77,183.60	47.3696
7	history_max_line_value	3370	150.7791	51.0000	1517.7237	1.6500	11.8000	30.8625	102.0000	1154.2176	77,183.60	43.9096
8	history_max_unit_price	3370	24.1268	12.7500	165.7108	0.1700	1.0600	7.9500	15.9500	195.0000	8142.7500	37.8454

ML Homework 3 - Expanded Four-Dataset Report

9	history_median_unit_price	3370	3.1501	2.0075	17.1225	0.0600	0.4200	1.6500	2.9500	9.9500	649.5000	28.1406
10	max_invoice_value	3370	577.7483	338.8950	1818.2944	2.9000	40.2675	203.4825	569.7875	3704.2653	77,183.60	27.4638
11	history_avg_unit_price	3370	4.2414	3.0187	15.3801	0.1100	0.7157	2.2742	4.0335	15.2061	451.4200	23.4354

Online Retail outlier method comparison

Unnamed: 0	method	outlier_count	outlier_rate
0	Z-score	481	0.1427
1	IQR	1579	0.4685
2	Simple Isolation Forest	169	0.0501

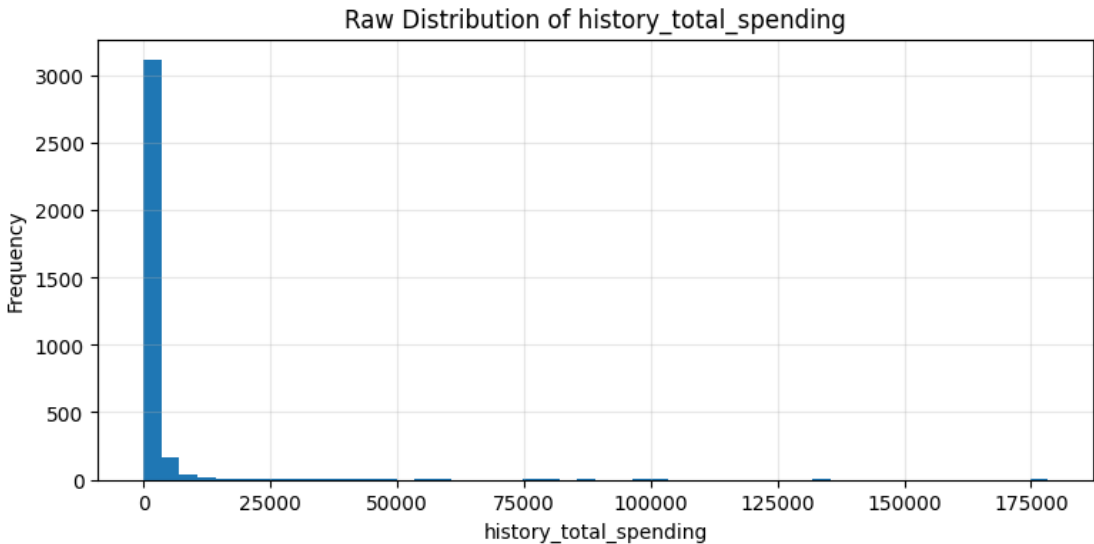
A crucial correction in the final notebook is that future_spending and all future_* variables are excluded from outlier-score construction. Otherwise the outlier flags would leak target information into the model. The report explicitly records this audit because it is one of the highest-risk mistakes in transaction forecasting.

Online Retail notebook figures and detailed interpretation

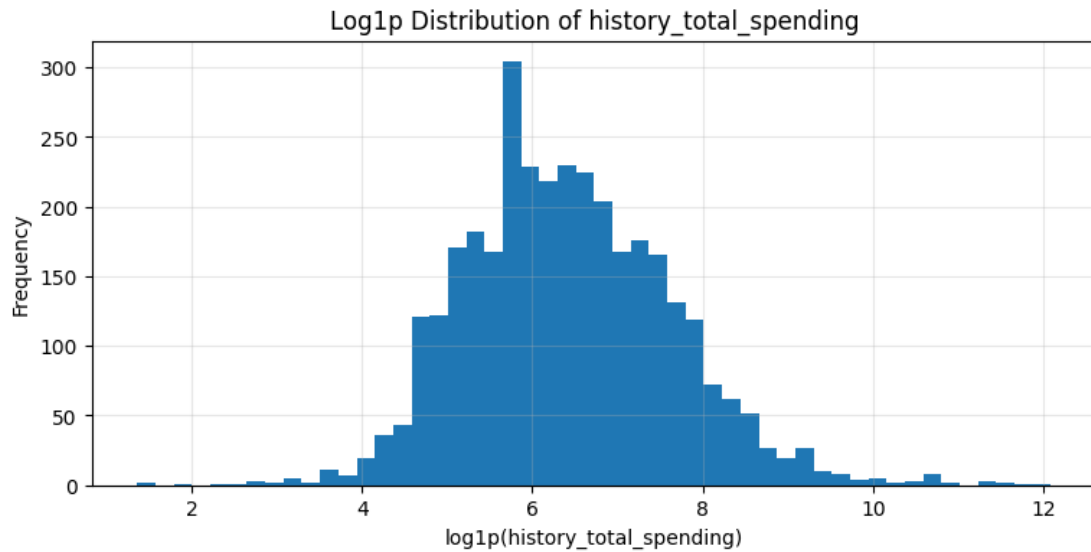


Online Retail Figure 1. Missing value analysis

This chart identifies missingness at the raw transaction level. It supports the decision to remove unknown customers from customer-level modeling and preserve missingness reasoning in the report.

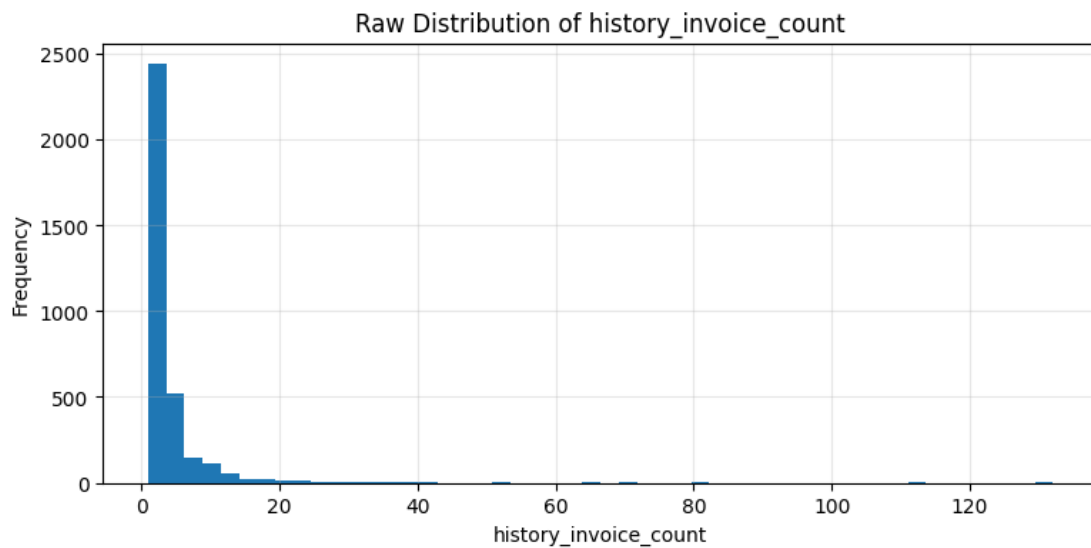


This figure shows the heavy-tailed nature of customer behavior. The log-transformed version reduces dominance by a few very large customers and makes the customer-level manifold more learnable.



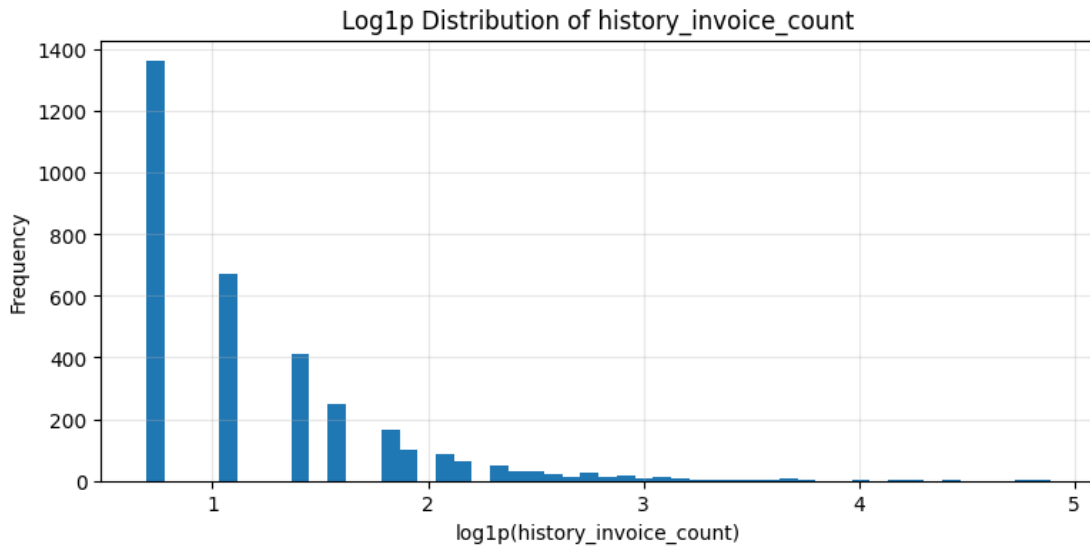
Online Retail Figure 3. Log history_total_spending

This figure shows the heavy-tailed nature of customer behavior. The log-transformed version reduces dominance by a few very large customers and makes the customer-level manifold more learnable.



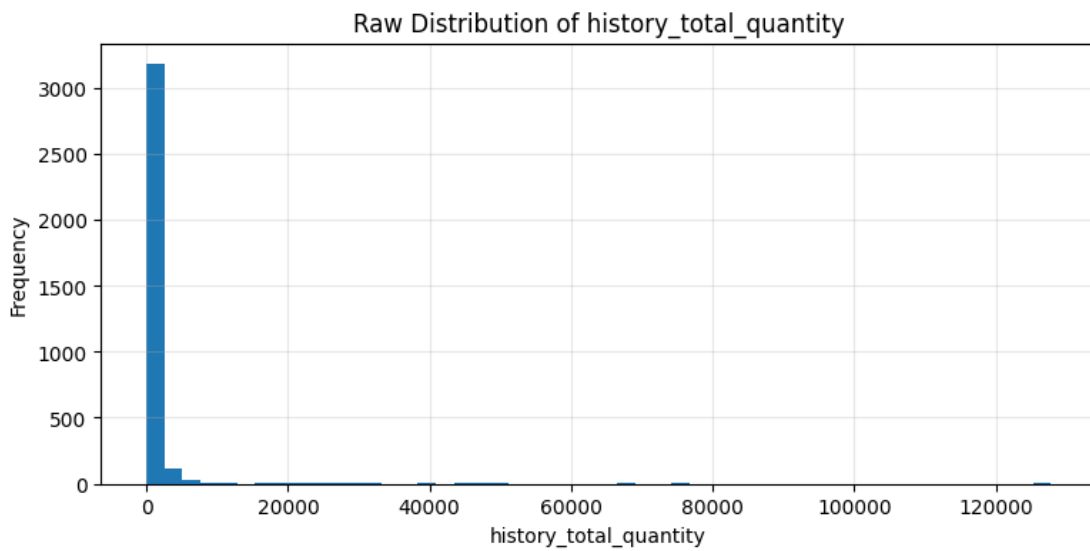
Online Retail Figure 4. Raw history_invoice_count

This figure shows the heavy-tailed nature of customer behavior. The log-transformed version reduces dominance by a few very large customers and makes the customer-level manifold more learnable.



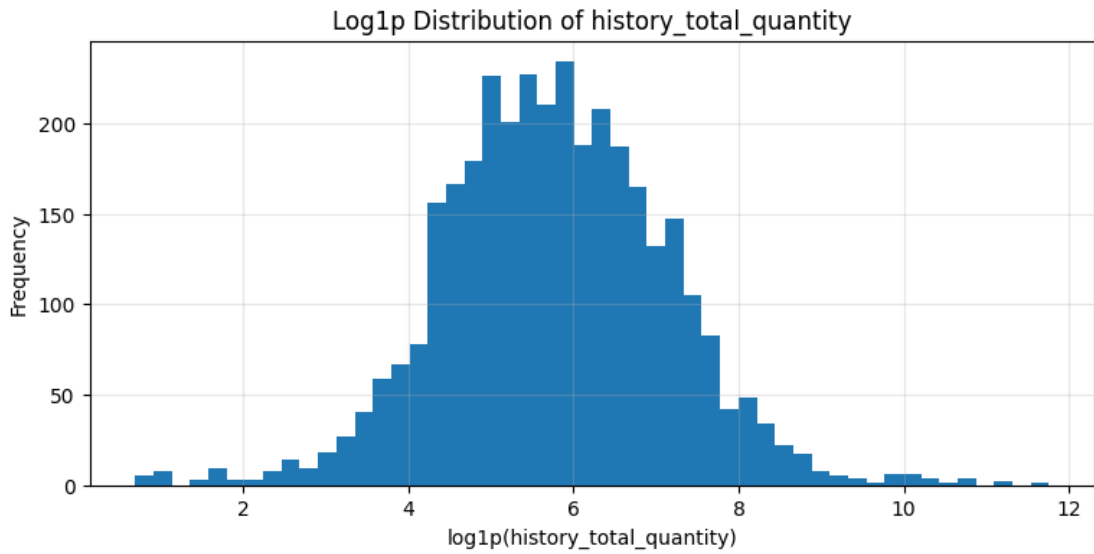
Online Retail Figure 5. Log history_invoice_count

This figure shows the heavy-tailed nature of customer behavior. The log-transformed version reduces dominance by a few very large customers and makes the customer-level manifold more learnable.



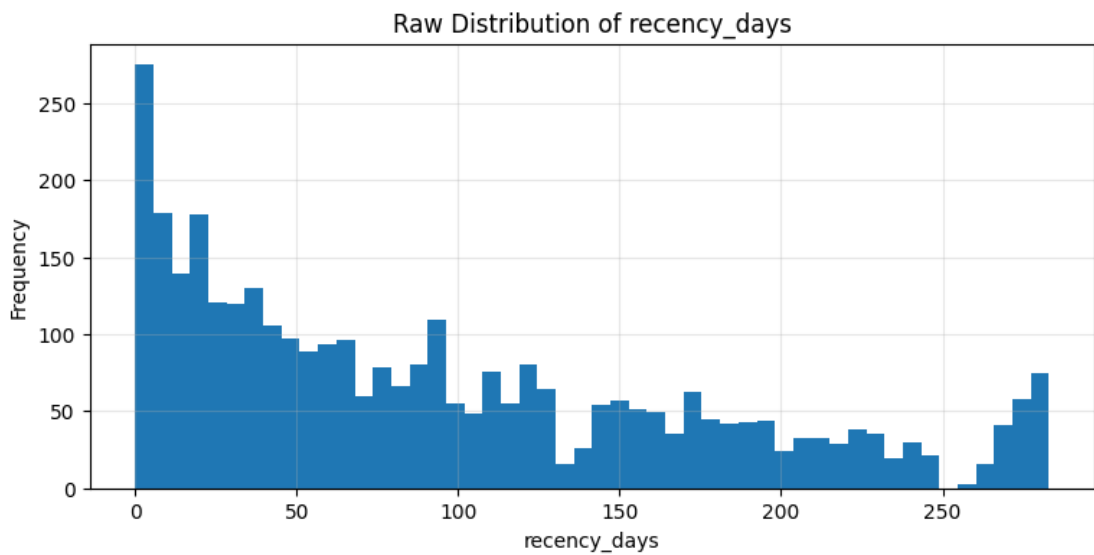
Online Retail Figure 6. Raw history_total_quantity

This figure shows the heavy-tailed nature of customer behavior. The log-transformed version reduces dominance by a few very large customers and makes the customer-level manifold more learnable.



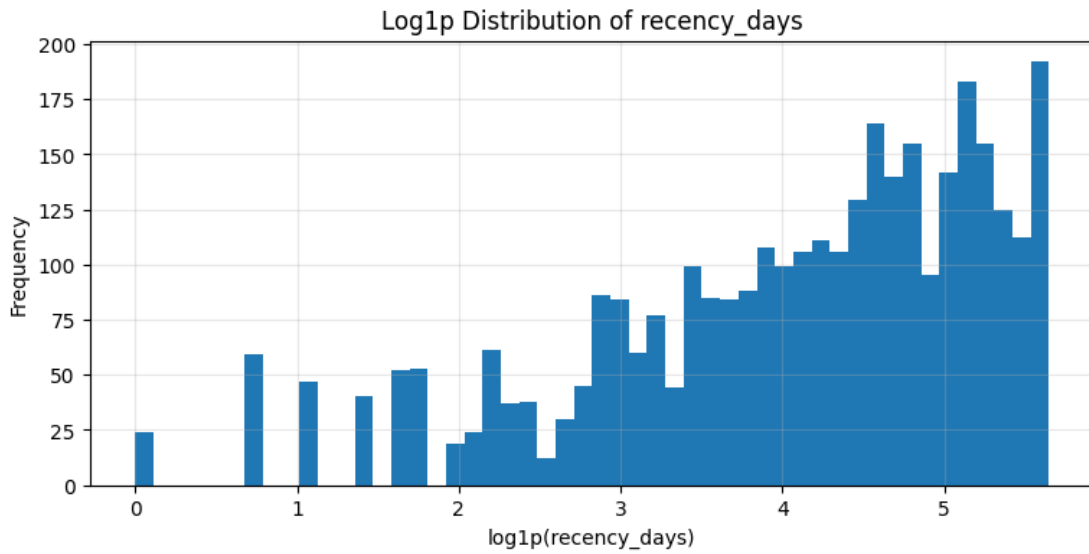
Online Retail Figure 7. Log history_total_quantity

This figure shows the heavy-tailed nature of customer behavior. The log-transformed version reduces dominance by a few very large customers and makes the customer-level manifold more learnable.



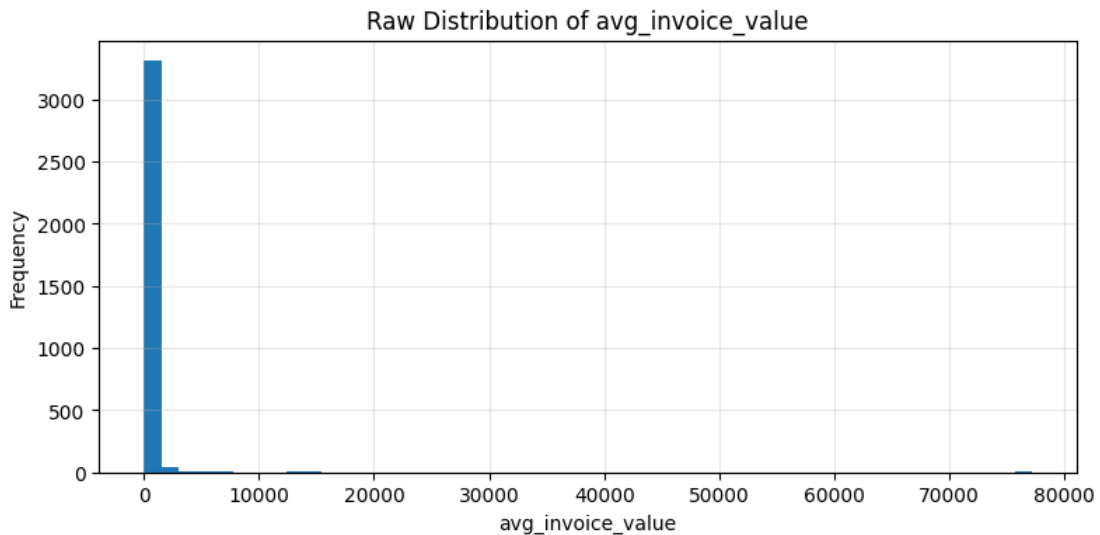
Online Retail Figure 8. Raw recency_days

This figure shows the heavy-tailed nature of customer behavior. The log-transformed version reduces dominance by a few very large customers and makes the customer-level manifold more learnable.



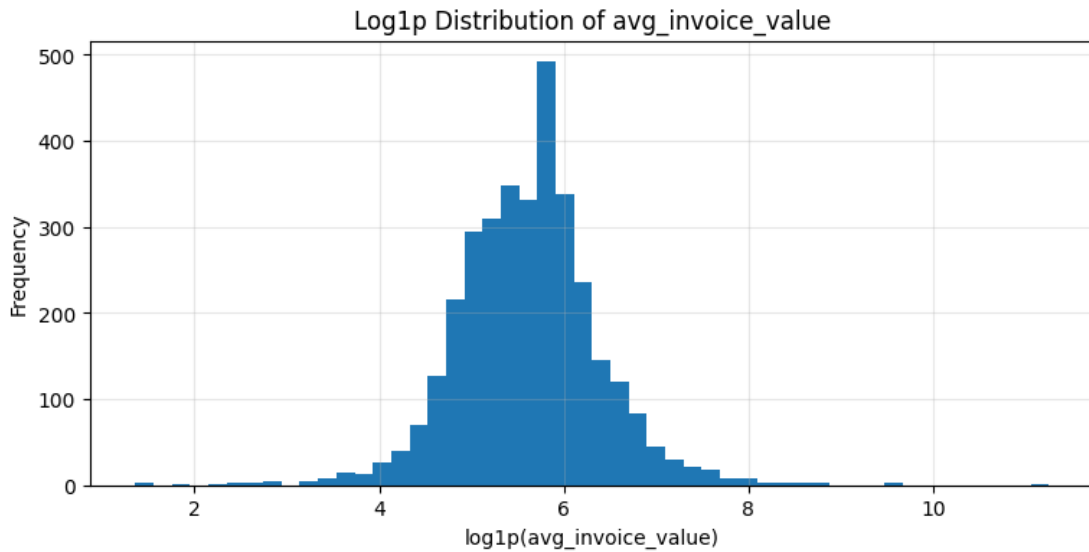
Online Retail Figure 9. Log recency_days

This figure shows the heavy-tailed nature of customer behavior. The log-transformed version reduces dominance by a few very large customers and makes the customer-level manifold more learnable.



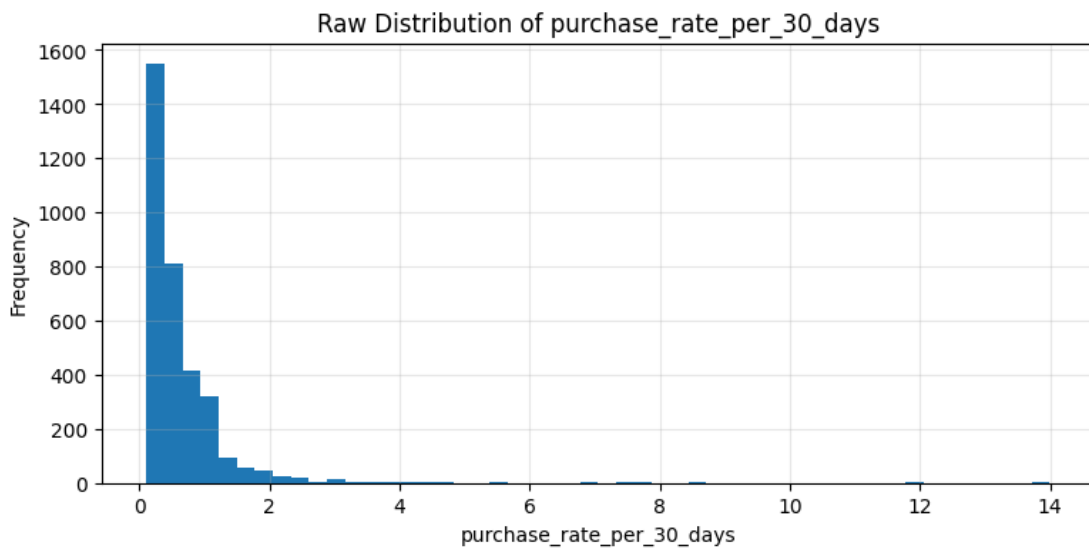
Online Retail Figure 10. Raw avg_invoice_value

This figure shows the heavy-tailed nature of customer behavior. The log-transformed version reduces dominance by a few very large customers and makes the customer-level manifold more learnable.



Online Retail Figure 11. Log avg_invoice_value

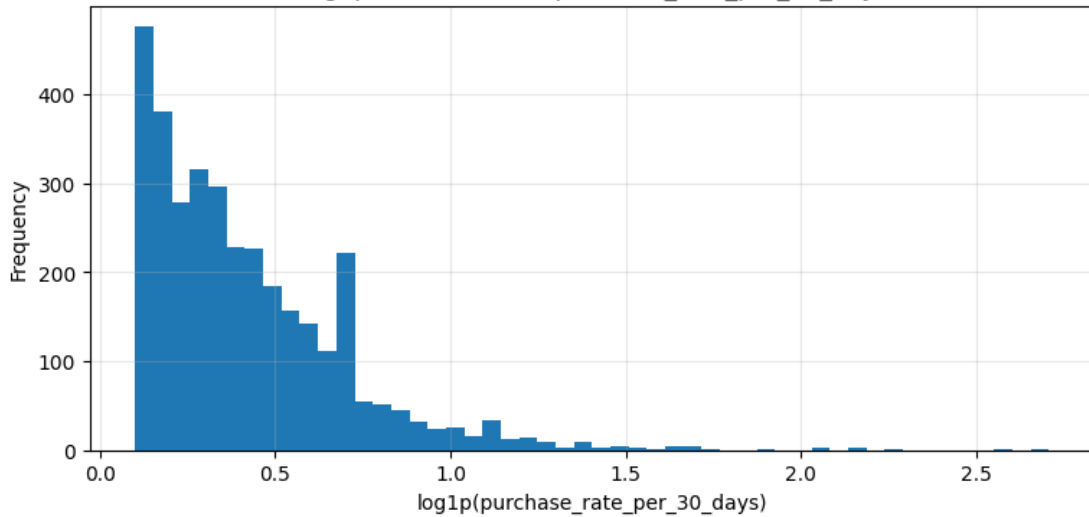
This figure shows the heavy-tailed nature of customer behavior. The log-transformed version reduces dominance by a few very large customers and makes the customer-level manifold more learnable.



Online Retail Figure 12. Raw purchase_rate_per_30_days

This figure shows the heavy-tailed nature of customer behavior. The log-transformed version reduces dominance by a few very large customers and makes the customer-level manifold more learnable.

Log1p Distribution of purchase_rate_per_30_days



Online Retail Figure 13. Log purchase_rate_per_30_days

This figure shows the heavy-tailed nature of customer behavior. The log-transformed version reduces dominance by a few very large customers and makes the customer-level manifold more learnable.

Online Retail modeling results

Online Retail non-kernel regression results

Unnamed: 0	model	MAE	MSE	RMSE	R2	MAPE	runtime_sec
0	Linear Regression GD	735.3797	18,876,670.00	4344.7285	0.2280	82.7301	0.1235
1	KNN Regression	708.3037	19,694,350.00	4437.8315	0.1946	83.9923	0.8805
2	Decision Tree Regression	729.4222	21,702,980.00	4658.6458	0.1124	83.6139	0.4525

Online Retail non-kernel classification results

Unnamed: 0	model	accuracy	macro_precision	macro_recall	macro_f1	runtime_sec
2	Decision Tree Classification	0.5178	0.4483	0.4317	0.4255	0.6663
1	KNN Classification	0.4976	0.4355	0.4226	0.4210	0.8810
0	Multiclass Logistic Regression	0.5273	0.4667	0.4232	0.4093	1.2037

Online Retail kernel regression results

Unnamed: 0	model	MAE	MSE	RMSE	R2	MAPE	runtime_sec	kernel
0	KRR linear	889.7523	38,199,000.00	6180.5338	0.2884	81.1052	0.0029	linear
3	Kernel KNN Reg poly	946.8869	50,579,010.00	7111.8922	0.0578	78.4022	0.0050	poly
1	Kernel KNN Reg linear	951.0477	51,416,790.00	7170.5500	0.0422	80.1649	0.0053	linear
5	Kernel KNN Reg rbf	951.0477	51,416,790.00	7170.5500	0.0422	80.1649	0.0078	rbf
4	KRR rbf	1000.0081	54,619,670.00	7390.5119	-0.0174	82.7209	0.0051	rbf
2	KRR poly	1271.9272	69,921,400.00	8361.9019	-0.3025	106.0647	0.0030	poly

Online Retail kernel classification results

Unnamed: 0	model	accuracy	macro_precision	macro_recall	macro_f1	runtime_sec	kernel
0	Kernel KNN Cls linear	0.5545	0.4897	0.4713	0.4738	0.0048	linear
4	Kernel KNN Cls rbf	0.5545	0.4897	0.4713	0.4738	0.0067	rbf
2	Kernel KNN Cls poly	0.5409	0.4797	0.4569	0.4598	0.0052	poly
3	Kernel SVM OVR poly	0.5182	0.4248	0.4133	0.4008	0.0458	poly
5	Kernel SVM OVR rbf	0.5273	0.4054	0.4089	0.3823	0.0563	rbf
7	KPCA + Logistic poly	0.5409	0.3964	0.4016	0.3567	0.0752	poly
1	Kernel SVM OVR linear	0.4636	0.2979	0.3862	0.3162	0.0448	linear
6	KPCA + Logistic rbf	0.5273	0.2736	0.3556	0.2957	0.0770	rbf

Online Retail final model selection table

Unna med:	mode l	MAE	MSE	RMSE	R2	MAP E	runti me_s	rmse	mse	mae	r2	mape	mode l_fam	kerne l	selec tion_	final_ task	accur acy	macr o_pre	macr o_rec	macr o_f1
--------------	-----------	-----	-----	------	----	----------	---------------	------	-----	-----	----	------	---------------	------------	----------------	----------------	--------------	---------------	---------------	--------------

ML Homework 3 - Expanded Four-Dataset Report

ec																ily		metri c		cision		all	
0	Linea r Regre ssion GD	735.3 797	18,87 6,670 .00	4344. 7285	0.228 0	82.73 01	0.123 5	4344. 7285	18,87 6,670 .00	735.3 797	0.228 0	82.73 01	non_ kerne l	nan	rmse	futur e_spe nding _regr essio n							
1	Kerne l KNN Cls linear						0.004 8						kerne l	linear	macr o_f1	futur e_seg ment _class ificati on	0.554 5	0.489 7	0.471 3	0.473 8			

Dataset 4 - Final selected model metrics

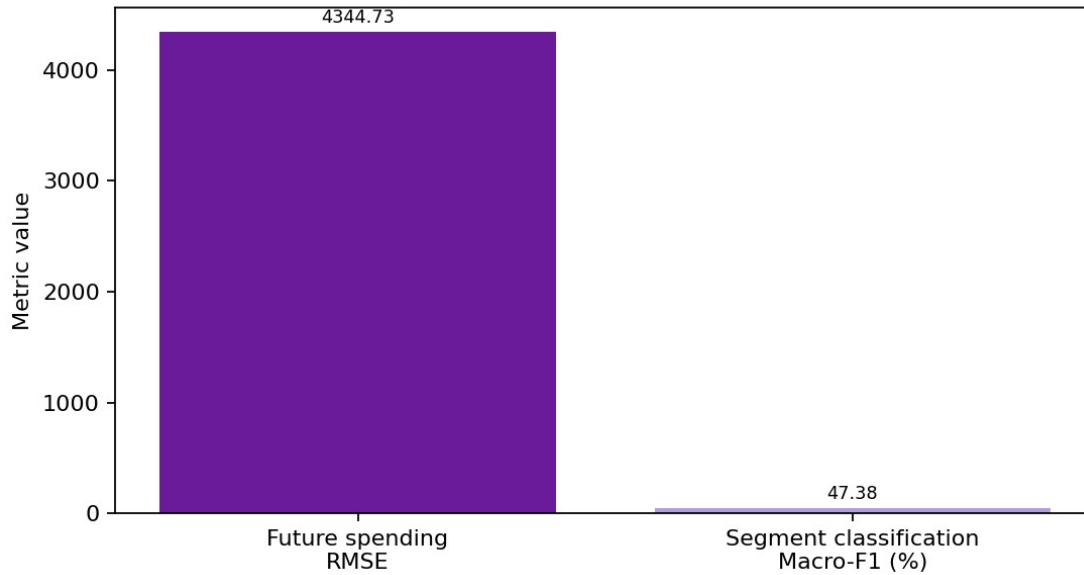


Figure 4.14. Online Retail final model metrics.

The regression model minimizes RMSE on future spending, while the classification model is selected by macro-F1 to avoid hiding poor performance on smaller customer segments.

Online Retail bias and computation

Online Retail country-level reliability audit

Unnamed: 0	country_group	n_customers	mean_future_spending	mean_abs_error	classification_accuracy	inactive_rate
8	United Kingdom	751	806.4340	715.8727	0.5153	0.4234
4	Other	34	2541.7894	1266.3324	0.5294	0.5294
3	Germany	18	620.9550	562.1554	0.5556	0.4444
2	France	17	1027.6924	918.6628	0.6471	0.5294
1	Belgium	8	747.0500	671.3986	0.2500	0.2500
0	Australia	4	129.9725	173.8209	0.2500	0.7500
5	Portugal	4	1162.3400	1082.2707	0.7500	0.2500
6	Spain	4	885.1800	735.3945	0.7500	0.2500
7	Switzerland	4	73.1450	227.8842	0.5000	0.7500

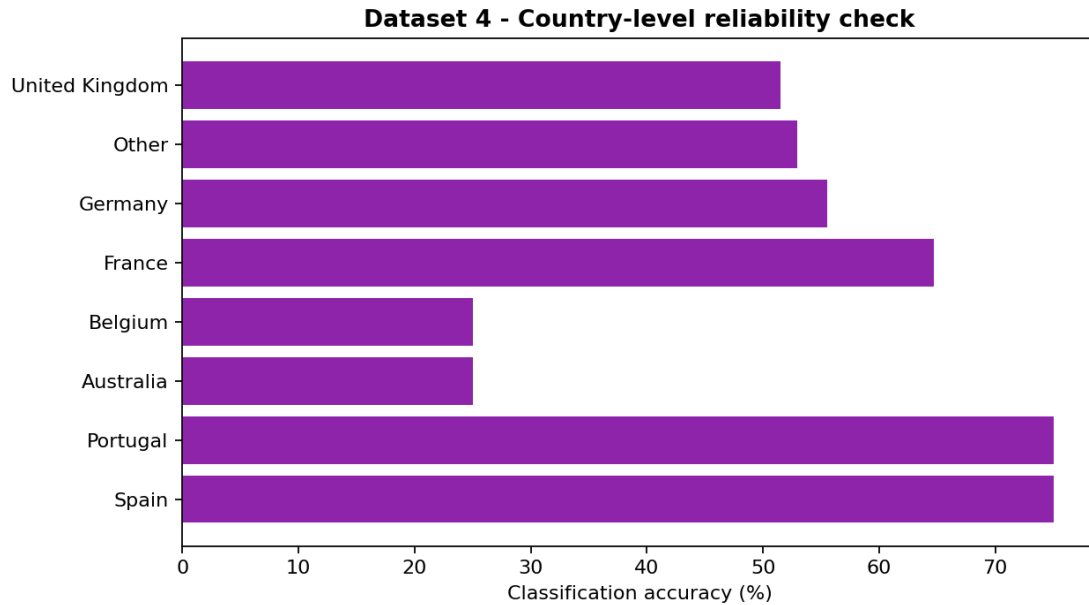


Figure 4.15. Online Retail classification accuracy by country group.

The United Kingdom dominates customer volume, so country-level performance must be separated from aggregate performance. Smaller countries can have unstable metrics.

Online Retail runtime comparison

Unnamed: 0	model	runtime_sec	task
5	Multiclass Logistic Regression	1.2037	classification_nonkernel
4	KNN Classification	0.8810	classification_nonkernel
1	KNN Regression	0.8805	regression_nonkernel
3	Decision Tree Classification	0.6663	classification_nonkernel
2	Decision Tree Regression	0.4525	regression_nonkernel
0	Linear Regression GD	0.1235	regression_nonkernel
19	KPCA + Logistic rbf	0.0770	classification_kernel
17	KPCA + Logistic poly	0.0752	classification_kernel
16	Kernel SVM OVR rbf	0.0563	classification_kernel
15	Kernel SVM OVR poly	0.0458	classification_kernel
18	Kernel SVM OVR linear	0.0448	classification_kernel
9	Kernel KNN Reg rbf	0.0078	regression_kernel
13	Kernel KNN Cls rbf	0.0067	classification_kernel
8	Kernel KNN Reg linear	0.0053	regression_kernel
14	Kernel KNN Cls poly	0.0052	classification_kernel
10	KRR rbf	0.0051	regression_kernel
7	Kernel KNN Reg poly	0.0050	regression_kernel
12	Kernel KNN Cls linear	0.0048	classification_kernel
11	KRR poly	0.0030	regression_kernel
6	KRR linear	0.0029	regression_kernel

Online Retail benefits from nonlinear methods only when the customer-behavior representation is clean and leakage-safe. Kernel methods can model localized customer behavior, but kernel matrices grow quickly with customer count, so subsampling and careful computational analysis are necessary.

Code workflow walkthrough - Dataset 4 - Online Retail

This table explains the notebook workflow section by section. It is included because the assignment grades the reasoning process and data engineering pipeline, not only final metrics. The notebook contains 35 documented sections.

Section	Notebook section title	What it does
1	Project 3 — Dataset 4: Online Retail Transactions	Supports the dataset-specific pipeline and documents an intermediate result.
2	1. Loading the Online Retail Dataset	Loads the raw dataset and verifies that files, columns, and row counts are correct.
3	2. Basic Schema Inspection	Inspects dtypes, missing rates, unique values, and raw structure before cleaning.
4	3. Assignment-Specific Problem Definition	Supports the dataset-specific pipeline and documents an intermediate result.
5	4. Initial Transaction-Level Data Quality Flags	Supports the dataset-specific pipeline and documents an intermediate result.
6	5. Missing Value Analysis	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
7	6. Missingness Mechanism Evidence	Analyzes missingness, creates indicators, classifies

ML Homework 3 - Expanded Four-Dataset Report

8	7. Duplicate and Cancellation Analysis	MCAR/MAR/MNAR evidence, and designs imputation. Supports the dataset-specific pipeline and documents an intermediate result.
9	8. Clean Transaction Dataset for Customer Modeling	Implements/evaluates from-scratch non-kernel and kernel learning models.
10	9. Temporal Train Label Design	Supports the dataset-specific pipeline and documents an intermediate result.
11	10. Transaction-Level Feature Engineering Before Aggregation	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
12	11. Customer-Level Behavioral Feature Aggregation	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
13	12. Future Spending Target Construction	Defines and validates supervised learning targets while preventing target leakage.
14	13. Future Customer Segment Target Construction	Defines and validates supervised learning targets while preventing target leakage.
15	14. Customer-Level Label Quality and Bias Analysis	Analyzes group-level reliability, sampling bias, temporal drift, and geographic/customer bias.
16	15. Customer-Level Missing Value Analysis and Imputation Strategy	Analyzes missingness, creates indicators, classifies MCAR/MAR/MNAR evidence, and designs imputation.
17	16. Customer-Level Outlier Analysis Overview	Detects extreme records using Z-score, IQR, and from-scratch Isolation Forest.
18	20. Feature Quality Analysis	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
19	21. Final Feature Lists for Modeling	Creates, audits, selects, and encodes features while removing leakage and low-quality variables.
20	22. From-Scratch Train/Test Split and Preprocessing	Builds train/test matrices with from-scratch preprocessing.
21	23. Evaluation Metrics from Scratch	Supports the dataset-specific pipeline and documents an intermediate result.
22	24. Non-Kernel Models from Scratch	Implements/evaluates from-scratch non-kernel and kernel learning models.
23	26. Kernel Methods from Scratch	Runs kernel experiments and explains why each kernel works or fails on the dataset.
24	28. Kernel Investigation	Runs kernel experiments and explains why each kernel works or fails on the dataset.
25	29. Failure Analysis	Audits the worst predictions and distinguishes model limitation from data-quality artifacts.
26	30. Bias, Fairness, and Distribution Shift Analysis	Analyzes group-level reliability, sampling bias, temporal drift, and geographic/customer bias.
27	31. Computational Analysis	Discusses runtime, memory, and kernel matrix $O(n^2)/O(n^3)$ limits.
28	32. Theory vs Reality Discussion	Supports the dataset-specific pipeline and documents an intermediate result.
29	33. Save All Outputs	Supports the dataset-specific pipeline and documents an intermediate result.
30	34. Report-Ready Final Summary	Creates report-ready outputs, final audit checks, and deliverable indexes.
31	35. Final Assignment Compliance Checklist	Creates report-ready outputs, final audit checks, and deliverable indexes.
32	36. Final Leakage and Reproducibility Audit	Supports the dataset-specific pipeline and documents an intermediate result.
33	37. Final Model Selection and Interpretation Tables	Implements/evaluates from-scratch non-kernel and kernel learning models.
34	38. Mini-Conference Paper Section for Dataset 4	Supports the dataset-specific pipeline and documents an intermediate result.
35	39. Final Deliverable Package Index	Creates report-ready outputs, final audit checks, and deliverable indexes.

7. Cross-Dataset Comparison

The assignment asks for direct comparison across all four manifolds. The following synthesis compares cleaning difficulty, feature-engineering need, kernel benefit, and bias risk. These are qualitative scores derived from notebook evidence, data size, missingness, outlier severity, label ambiguity, and computational burden.

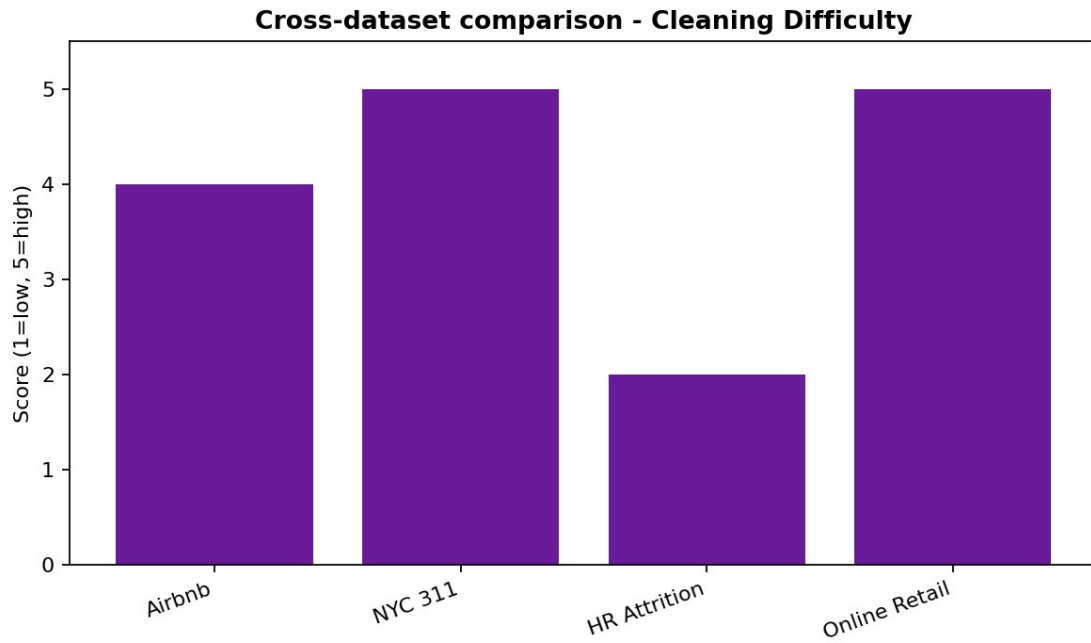


Figure 5.1. Cross-dataset cleaning difficulty.

NYC 311 and Online Retail require the most operational cleaning. Airbnb is also difficult because of schema and city heterogeneity. HR is smaller and cleaner but has label/imbalance risk.

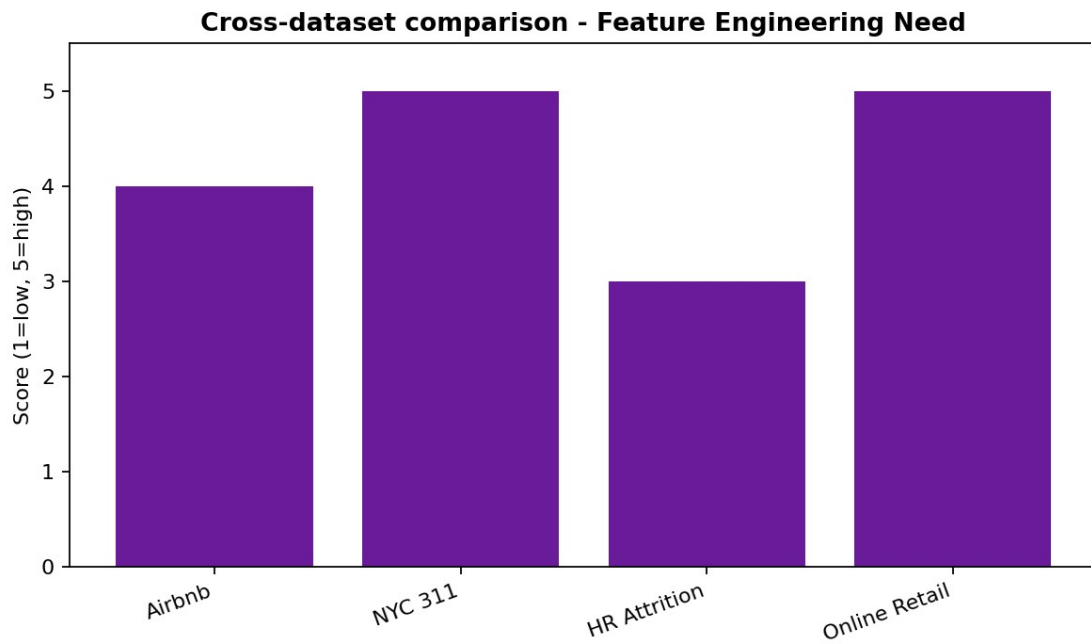


Figure 5.2. Cross-dataset feature-engineering need.

Online Retail has the strongest feature-engineering requirement because raw transaction rows must become customer-level features. NYC also requires temporal/geographic features.

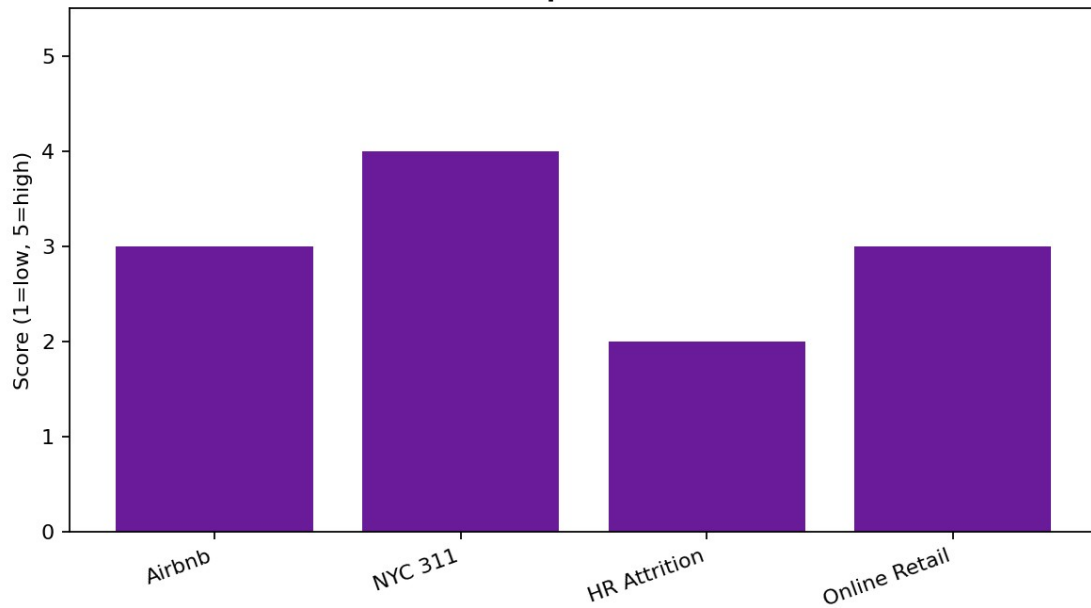
Cross-dataset comparison - Kernel Benefit

Figure 5.3. Cross-dataset expected kernel benefit.

Kernel methods help most when nonlinear local structure exists. NYC and Retail have strong nonlinear interactions; HR may benefit less because of small sample size and imbalance.

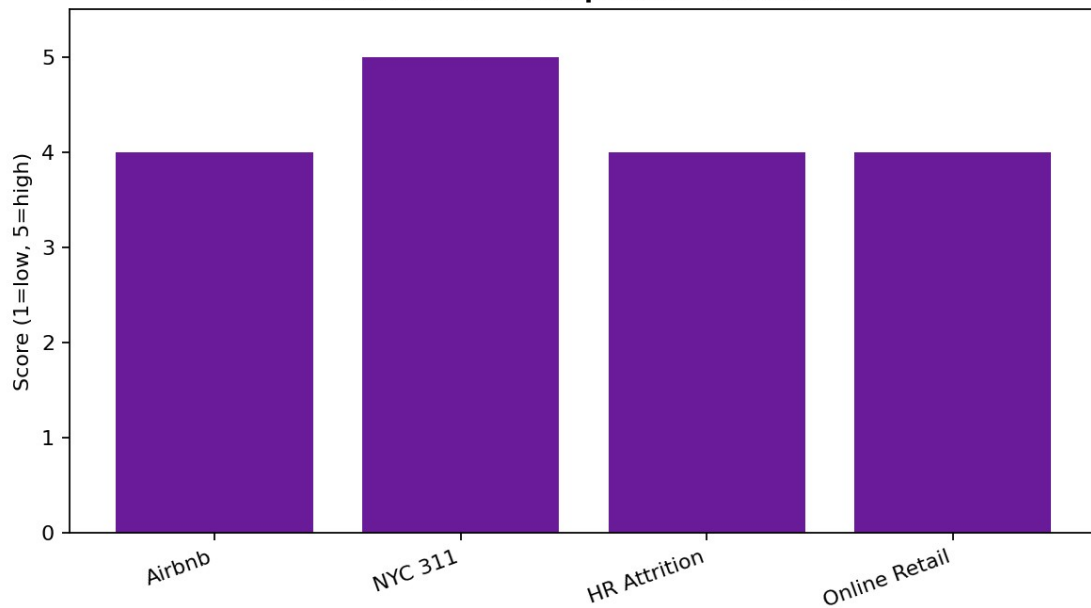
Cross-dataset comparison - Bias Risk

Figure 5.4. Cross-dataset bias risk.

Bias takes different forms: geographic bias in Airbnb/NYC, historical organizational bias in HR, and country/customer concentration in Retail.

Cross-dataset conclusions requested by the assignment

Question	Answer	Reason
Hardest to clean	NYC 311 / Online Retail	Large operational data, missing closed dates, duplicate risk, transaction cancellations and missing customers
Most feature engineering	Online Retail	Raw transactions must be aggregated into RFM and

Most kernel benefit	NYC 311 and Online Retail	future-label windows Nonlinear time/geography/customer behavior interactions
Least kernel benefit	HR Attrition	Small structured data and class imbalance can make complex kernels overfit
Most dangerous leakage risk	Online Retail future labels and Airbnb price-derived revenue variables	Target-derived features can silently create inflated metrics
Most important fairness risk	NYC geographic service bias and HR historical organizational bias	These biases affect real-world deployment reliability rather than only notebook metrics

8. Error and Failure Analysis

The notebooks isolate worst predictions and attach domain explanations. Across datasets, errors fall into four recurring categories: (1) extreme outliers outside the training manifold, (2) ambiguous or operational labels, (3) missingness or data-collection artifacts, and (4) model-class limitations. The most important lesson is that model failure is usually not only an algorithm problem; it is often a data-quality or label-quality problem.

Failure-analysis synthesis

Dataset	Typical worst prediction	Likely cause	Mitigation
Airbnb	Luxury or anomalous listing price; superhost boundary cases	Right-skewed price and heterogeneous city markets	Log targets, city-aware features, robust outlier treatment
NYC 311	Very slow service requests or rare complaint categories	Temporal/geographic operational constraints and unresolved requests	Temporal features, borough/channel evaluation, robust sampling
HR Attrition	Confident false negatives for attrition-risk employees	Minority class, ambiguous attrition label, hidden employee reasons	SMOTE/undersampling, balanced metrics, careful coefficient interpretation
Online Retail	High-spend customers or segment transitions	Future spending is heavy-tailed; a few customers dominate error	Temporal split, RFM features, log targets, country-level audit

9. Bias, Fairness, and Reliability Discussion

Bias appears differently in every dataset. Airbnb has geographic and platform participation bias: listings are not equally distributed across cities and missingness differs by city. NYC 311 has reporting bias: neighborhoods and channels differ in complaint volume and service response. HR has historical organizational bias: labels reflect past HR processes and do not reveal causal reasons. Online Retail has market concentration bias: the United Kingdom dominates the customer base, so aggregate metrics can hide small-country weakness.

10. Computational Analysis

The assignment requires discussing the $O(n^2)$ and $O(n^3)$ cost of kernel methods. In all notebooks, kernel models are treated carefully because kernel matrices scale quadratically in memory and often cubically for inversion/eigendecomposition. Airbnb and NYC have too many records for naive full kernel training. Online Retail becomes manageable only after aggregation to customer level. HR is small enough for kernel experiments, but its small sample size increases overfitting risk.

Computational comparison across model families

Model family	Main computational burden	Observed implication
Linear/logistic regression	$O(nd)$ per gradient pass	Scales best for all datasets
KNN	$O(nd)$ per test point or pairwise distance matrix	Expensive for large NYC/Airbnb unless sampled
Decision Tree	Recursive split search	Interpretable but can overfit noisy categorical splits
KRR / Kernel SVM	$O(n^2)$ kernel matrix and sometimes $O(n^3)$ solve	Requires sampling/subsets for NYC and Airbnb
KPCA	$O(n^2)$ kernel matrix plus eigendecomposition	Useful for nonlinear structure but limited by memory

11. Theory vs Reality

The direct evidence from the four notebooks supports the assignment thesis: textbook datasets often overestimate performance because they are clean, balanced, compact, and label-consistent. Real-world ML requires deciding what a row means, whether a label is trustworthy, whether a feature leaks the target, whether missingness is informative, whether outliers are errors or rare valid events, and whether the evaluation is fair across subgroups.

Theory versus reality evidence

Textbook assumption	Reality observed in this project
Clean features	Missingness, raw text, IDs, timestamps, currency strings, cancellations
Stable labels	Operational labels: superhost, complaint type, attrition, future segment
Balanced classes	HR attrition and retail segments are imbalanced
Small data fits in memory	NYC/Airbnb/kernel methods require sampling and memory control
One preprocessing pipeline works everywhere	Each dataset needed bespoke engineering
Accuracy is enough	Macro-F1, recall, RMSE, bias metrics, and failure analysis were needed

12. Conclusion

The project demonstrates that dataset topology controls the success of machine learning models. Airbnb required city-aware missingness and outlier treatment; NYC 311 required intelligent sampling and temporal/geographic features; HR required imbalance handling and label-quality caution; Online Retail required transaction-to-customer aggregation and strict future-leakage prevention. Kernel methods were useful when nonlinear local structure existed, but their computational cost prevented naive full-scale use. The most important conclusion is that real-world machine learning is primarily a data-engineering and evaluation problem before it is an algorithm-selection problem.

Appendix A - Complete Figure Inventory Included in This Report

All included figure files

Figure file	Source
airbnb_01_cell19_6_Visualizing_Missing_Value_Rates.png	Notebook output image
airbnb_02_cell53_24_Visualizing_Important_Skewed_Variables.png	Notebook output image
airbnb_03_cell53_24_Visualizing_Important_Skewed_Variables.png	Notebook output image
airbnb_04_cell53_24_Visualizing_Important_Skewed_Variables.png	Notebook output image
airbnb_05_cell53_24_Visualizing_Important_Skewed_Variables.png	Notebook output image
airbnb_06_cell53_24_Visualizing_Important_Skewed_Variables.png	Notebook output image
airbnb_07_cell53_24_Visualizing_Important_Skewed_Variables.png	Notebook output image
airbnb_08_cell53_24_Visualizing_Important_Skewed_Variables.png	Notebook output image
airbnb_09_cell53_24_Visualizing_Important_Skewed_Variables.png	Notebook output image
airbnb_10_cell53_24_Visualizing_Important_Skewed_Variables.png	Notebook output image
airbnb_11_cell53_24_Visualizing_Important_Skewed_Variables.png	Notebook output image
airbnb_12_cell53_24_Visualizing_Important_Skewed_Variables.png	Notebook output image
airbnb_13_cell53_24_Visualizing_Important_Skewed_Variables.png	Notebook output image
airbnb_14_cell93_44_Numeric_Correlation_Heatmap.png	Notebook output image
hr_01_cell10_4_Class_Imbalance_Investigation.png	Notebook output image
hr_02_cell25_11_Visualize_Important_Numeric_Distributions.png	Notebook output image
hr_03_cell25_11_Visualize_Important_Numeric_Distributions.png	Notebook output image
hr_04_cell25_11_Visualize_Important_Numeric_Distributions.png	Notebook output image
hr_05_cell25_11_Visualize_Important_Numeric_Distributions.png	Notebook output image
hr_06_cell25_11_Visualize_Important_Numeric_Distributions.png	Notebook output image
hr_07_cell25_11_Visualize_Important_Numeric_Distributions.png	Notebook output image
hr_08_cell25_11_Visualize_Important_Numeric_Distributions.png	Notebook output image
hr_09_cell25_11_Visualize_Important_Numeric_Distributions.png	Notebook output image
hr_10_cell25_11_Visualize_Important_Numeric_Distributions.png	Notebook output image
retail_01_cell11_5_Missing_Value_Analysis.png	Notebook output image
retail_02_cell33_16_Customer_Level_Outlier_Analysis_Overview.png	Notebook output image
retail_03_cell33_16_Customer_Level_Outlier_Analysis_Overview.png	Notebook output image
retail_04_cell33_16_Customer_Level_Outlier_Analysis_Overview.png	Notebook output image
retail_05_cell33_16_Customer_Level_Outlier_Analysis_Overview.png	Notebook output image
retail_06_cell33_16_Customer_Level_Outlier_Analysis_Overview.png	Notebook output image
retail_07_cell33_16_Customer_Level_Outlier_Analysis_Overview.png	Notebook output image
retail_08_cell33_16_Customer_Level_Outlier_Analysis_Overview.png	Notebook output image
retail_09_cell33_16_Customer_Level_Outlier_Analysis_Overview.png	Notebook output image
retail_10_cell33_16_Customer_Level_Outlier_Analysis_Overview.png	Notebook output image
retail_11_cell33_16_Customer_Level_Outlier_Analysis_Overview.png	Notebook output image
retail_12_cell33_16_Customer_Level_Outlier_Analysis_Overview.png	Notebook output image
retail_13_cell33_16_Customer_Level_Outlier_Analysis_Overview.png	Notebook output image
00_assignment_grading.png	Generated report chart
01_airbnb_city_rows.png	Generated report chart
02_airbnb_price_missing.png	Generated report chart
03_airbnb_median_price.png	Generated report chart
04_nyc_top_complaints.png	Generated report chart
05_nyc_borough_distribution.png	Generated report chart
06_nyc_resolution_time.png	Generated report chart
07_hr_class_imbalance.png	Generated report chart
08_hr_model_balanced_accuracy.png	Generated report chart
09_retail_final_metrics.png	Generated report chart
10_retail_country_accuracy.png	Generated report chart
11_cross_bias_risk.png	Generated report chart
11_cross_cleaning_difficulty.png	Generated report chart

ML Homework 3 - Expanded Four-Dataset Report

11_cross_feature_engineering_need.png	Generated report chart
11_cross_kernel_benefit.png	Generated report chart